



ROYAUME DU MAROC
UNIVERSITE ABDELMALEK ESSAÂDI
FACULTE DES SCIENCES ET TECHNIQUES
DEPARTEMENT GENIE INFORMATIQUE



Master en Intelligence Artificielle et Science des Données



Module: Blockchain

**Traffic Core : Simulation et coordination du trafic urbain
dans une Smart City décentralisée via la blockchain**

Réalisé par :

LAAKEL GAUZI SOUMAYA

CHELLEY KHOULOU

IBN EL MOKADEM ZINEB

KHALIFI AYA

Année Universitaire : 2025/2026

Table des matières

Listes des Figures	5
Listes des Tableaux	6
Liste des acronymes	7
1. INTRODUCTION	10
1.1. Contexte des Smart Cities	11
1.1.1. Évolution et besoins des villes modernes.....	11
1.1.2. Les composantes clés des Smart Cities.....	11
1.1.3. Objectifs des Smart Cities	12
1.2. Problématiques des architectures centralisées	13
1.2.1. Point de défaillance unique	13
1.2.2. Problèmes de sécurité et de confidentialité	13
1.2.3. Manque de transparence et de contrôle	14
1.2.4. Manque de scalabilité.....	14
1.2.5. Problèmes de résilience et d'indépendance	14
1.3. Présentation du projet de Smart City décentralisée	14
1.4. Spécifications techniques du module Traffic Core	15
1.5. Objectifs du projet Traffic Core	17
1.6. Objectifs spécifiques du projet	18
2. Fondements Théoriques et État de l'Art.....	21
2.1. Introduction à la blockchain.....	22
2.1.1. Fonctionnement de la Blockchain.....	22
2.1.2. Types de Blockchain.....	23
2.2. Hyperledger Fabric : Vue d'ensemble	24
2.2.1. Architecture d'Hyperledger Fabric.....	24
2.3. Les Smart Contracts et Chaincode dans Fabric	25
2.3.1. Fonctionnement des Smart Contracts	25
2.4. Les Smart Cities et l'intégration des technologies blockchain	25
2.4.1. Applications de la Blockchain dans les Smart Cities	25
2.5. Mécanismes de Consensus dans Hyperledger Fabric	26
2.5.1. Types de consensus dans Hyperledger Fabric	26
3. Conception du Système Traffic Core.....	29
3.1. Vue d'ensemble du module Traffic Core.....	30

3.2.	Architecture réseau Hyperledger Fabric	30
3.3.	Modélisation des données du trafic et IoT	33
3.3.1.	Données des routes et intersections	34
3.3.2.	Capteurs IoT	34
3.3.3.	Véhicules et conducteurs	34
3.3.4.	Relations entre les entités	35
3.3.5.	Intégration avec Hyperledger Fabric	35
3.4.	Conception des smart contracts.....	36
3.5.	Architecture de l'application simulateur / tableau de bord	38
4.	Implémentation et Déploiement de la Plateforme	42
4.1.	Environnement technique	43
4.2.	Scripts d'installation et d'orchestration	43
4.2.1.	Scripts d'installation (préparer l'environnement).....	43
4.2.2.	Scripts d'orchestration (démarrer/arrêter tout le projet)	43
4.2.3.	Script d'orchestration du réseau blockchain (Fabric).....	44
4.3.	Déploiement des chaincodes en mode CCaaS	45
4.4.	Implémentation détaillée des smart contracts	46
4.4.1.	Chaincode road-manager (gestion des routes & trafic)	46
4.4.2.	Chaincode sensor-data (capteurs IoT & lectures temps réel)	46
4.4.3.	Chaincode traffic-registry (véhicules, conducteurs, infractions).....	47
4.5.	Fonctionnement du simulateur et du tableau de bord	49
4.5.1.	Démarrage et accès à la plateforme	49
4.5.2.	Simulateur IoT (génération des données trafic).....	49
4.5.3.	Tableau de bord React (visualisation + contrôle)	50
4.5.4.	Mise à jour temps réel et intégration blockchain.....	50
5.	Architecture de Sécurité et SOC Intelligent.....	51
5.1.	Exigences de sécurité pour Traffic Core	52
5.1.1.	Confidentialité et sécurisation des communications.....	52
5.1.2.	Authentification et gestion des identités.....	52
5.1.3.	Autorisation et contrôle des transactions (policies)	52
5.1.4.	Intégrité et traçabilité des données	53
5.1.5.	Disponibilité et résilience (attaque, surcharge, pannes).....	53
5.1.6.	Surveillance et réponse aux incidents (SOC)	53
5.1.7.	Analyse intelligente (LLM) pour qualification des alertes	53

5.1.8.	Pilotage et vérification (API + dashboard)	54
5.2.	Architecture du Security Operations Center (SOC).....	54
5.2.1.	Objectifs du SOC dans Traffic Core.....	54
5.2.2.	Composants de l'architecture SOC (4 agents).....	55
5.2.3.	Intégration de l'IA (LLM) dans l'Analyzer	55
5.2.4.	Interfaces de contrôle (API + Dashboard).....	56
5.3.	Intégration du SOC avec Hyperledger Fabric.....	57
5.3.1.	Points d'intégration (où le SOC se branche)	57
5.3.2.	Journalisation (audit) des incidents sur la blockchain	58
5.3.3.	Détection d'événements liés à Fabric / réseau (exemple concret).....	58
5.3.4.	Interaction SOC ↔ réponse active (actions défensives)	58
5.3.5.	Intégration IA (LLM) pour enrichir la sécurité.....	59
5.4.	Modèle de menaces et attaques simulées	59
5.4.1.	Détection et réponse aux attaques.....	61
5.4.2.	Cartographie des attaques vers les composants SOC	61
6.	Expérimentation, Consensus et Résultats	62
6.1	Méthodologie expérimentale.....	63
6.2	Comparaison PoA vs PBFT	63
6.2.1	Latence moyenne	63
6.2.2	Débit maximal (TPS)	64
6.2.3	Résilience aux fautes	64
6.2.4	Consommation des ressources.....	64
6.3	Résultats et analyse	65
6.3.1	Choix du consensus selon le cas d'usage.....	65
6.3.2	Impact sur Traffic Core.....	65
6.4	Captures d'écran de l'application	65
6.4.1	Tableau de bord principal	65
6.4.2	Page de simulation du trafic	67
6.4.3	Page de gestion du réseau routier.....	69
6.4.4	Monitoring des capteurs IoT	70
6.4.5	Registre du trafic sur blockchain.....	71
6.4.6	Security Operations Center (SOC) intelligent	72
6.4.7	Tests et comparaison des mécanismes de consensus	75
7.	Conclusion	78

Annexe A : Configuration et Déploiement du Réseau Blockchain	80
AXE 1 : Configuration du Réseau Hyperledger Fabric.....	80
1.1 Définition des Organisations	80
1.2. Configuration de l'Orderer (etcdraft).....	80
1.3. Profils de Configuration des Canaux.....	81
AXE 2 : Génération des Artefacts du Réseau	82
2.1. Variables d'Environnement.....	82
2.2. Génération du Bloc de Genesis	82
2.3. Création des Transactions de Canal.....	82
2.4. Configuration des Anchor Peers	82
AXE 3 : Déploiement et Gestion des Canaux.....	83
3.1. Variables d'Environnement pour l'Orderer	83
3.2. Création du Canal Principal	83
3.3. Jointure des Peers au Canal.....	83
AXE 4 : Déploiement du Chaincode.....	85
4.1. Packaging du Chaincode	85
4.2. Installation sur les Peers	85
4.3. Approbation des Organisations	85
4.4. Vérification du Readiness Check.....	86
4.5. Commit du Chaincode	86
4.6. Vérification du Commit.....	87
AXE 5 : Tests Fonctionnels du Chaincode	87
5.1. Tests de Gestion des Routes	87
5.2. Requêtes de Consultation.....	88
5.3. Tests de Gestion des Véhicules.....	89
5.4. Vérification de la Base de Données	90

Listes des Figures

Figure 1: Schéma de l'architecture typique d'une Smart City	11
Figure 2: Schéma du projet de Smart City décentralisée avec Hyperledger Fabric.....	15
Figure 3:Schéma de fonctionnement d'une blockchain	22
Figure 4: Architecture d'Hyperledger Fabric	25
Figure 5:Schéma des mécanismes de consensus dans Hyperledger Fabric (Raft, PBFT, etc.)	28
Figure 6 : Architecture réseau Traffic.....	33
Figure 7: Diagramme UML des entités route, capteur, véhicule, conducteur	36
Figure 8:Diagramme des interactions entre les smart contracts	38
Figure 9:Schéma de l'architecture de l'application simulateur / tableau de bord	41
Figure 10: Diagramme de séquence d'une transaction CreateRoad.....	48
Figure 11: Diagramme de séquence d'une transaction RecordReading	48
Figure 12: Architecteur de SOC	54
Figure 13: Architecture du SOC (Defender / Controller / Sensor / Analyzer).....	57
Figure 14: Cartographie des attaques vers composants SOC	61
Figure 15: Tableau de bord - Métriques et statistiques.....	66
Figure 16: Tableau de bord - Visualisations détaillées	66
Figure 17:Simulation - Panneau de contrôle principal	67
Figure 18:Simulation - Carte interactive du trafic urbain	67
Figure 19:Simulation - Tableau de bord détaillé.....	68
Figure 20: Routes - Vue d'ensemble du réseau routier	69
Figure 21: Routes - Détail des intersections intelligentes.....	69
Figure 22: Capteurs - Réseau IoT de CASABLANCA	70
Figure 23: Registry - Gestion des véhicules	71
Figure 24: Registry - Gestion des conducteurs	71
Figure 25: Registry - Détection et suivi des violations.....	72
Figure 26: Security - Tableau de bord SOC	72
Figure 27: Security - Simulation d'attaques.....	73
Figure 28: Security - Résultats d'analyse d'attaque.....	74
Figure 29: Security - Gestion des alertes.....	74
Figure 30: Security - Journaux de sécurité temps réel	75
Figure 31: Consensus - Configuration des topologies	75
Figure 32: Consensus - Résultats comparatifs détaillés.....	76
Figure 33: Consensus - Recommandations d'utilisation.....	76
Figure 34: Consensus - Visualisation graphique des résultats.....	77

Listes des Tableaux

Tableau 1:Résumé des fonctionnalités du projet Traffic Core	17
Tableau 2:Exemples d'applications de la blockchain dans les Smart Cities.....	26
Tableau 3: Description des organisations : TrafficAuthority, MobilityServices, SensorNetwork.....	33
Tableau 4:Modèle des routes et intersections	34
Tableau 5: Modèle des capteurs IoT.....	34
Tableau 6:Modèle des véhicules et conducteurs.....	35
Tableau 7:Scripts d’installation et d’orchestration (Traffic Core).....	45
Tableau 8: Fonctions principales des chaincodes déployés.....	45
Tableau 9: Fonctions du chaincode road-manager	46
Tableau 10: Fonctions du chaincode sensor-data	47
Tableau 11: Fonctions du chaincode traffic-registry	48

Liste des acronymes

Acronyme	Signification (Français)	Signification (Anglais)	Définition dans le contexte
API	Interface de Programmation d'Applications	Application Programming Interface	Interface permettant la communication entre les composants du système
CCaaS	Chaincode en tant que Service	Chaincode as a Service	Mode de déploiement des smart contracts dans Hyperledger Fabric
CPU	Unité Centrale de Traitement	Central Processing Unit	Processeur, unité de calcul des serveurs
DDoS	Attaque par Déni de Service Distribué	Distributed Denial of Service	Attaque visant à rendre un service indisponible par saturation
Fabric	-	Hyperledger Fabric	Plateforme blockchain permissionnée utilisée dans le projet
GES	Gaz à Effet de Serre	Greenhouse Gases	Polluants environnementaux mesurés dans les Smart Cities
Go	-	Go (langage de programmation)	Langage utilisé pour développer les smart contracts
HTTPS	Protocole de Transfert HyperTexte Sécurisé	HyperText Transfer Protocol Secure	Protocole de communication sécurisée
IA	Intelligence Artificielle	Artificial Intelligence	Techniques d'apprentissage automatique utilisées dans le SOC
IoT	Internet des Objets	Internet of Things	Réseau de capteurs connectés collectant des données
IP	Protocole Internet	Internet Protocol	Adresse réseau unique identifiant un dispositif
JSON	Notation Objet JavaScript	JavaScript Object Notation	Format de données léger pour l'échange d'informations
JWT	Jeton Web JSON	JSON Web Token	Standard pour les tokens d'authentification sécurisés
LLM	Modèle de Langage Large	Large Language Model	Modèle d'IA comme Mistral utilisé pour l'analyse de sécurité

MAD	Dirham Marocain	Moroccan Dirham	Devise utilisée pour le calcul des amendes
MSP	Fournisseur de Services de Membres	Membership Service Provider	Gestion des identités dans Hyperledger Fabric
Node.js	-	Node.js	Environnement d'exécution JavaScript côté serveur
ONU	Organisation des Nations Unies	United Nations	Organisation internationale citée dans le contexte des Smart Cities
PBFT	Tolérance aux Fautes Byzantines Pratique	Practical Byzantine Fault Tolerance	Algorithme de consensus testé dans le projet
PoA	Preuve d'Autorité	Proof of Authority	Algorithme de consensus testé dans le projet
PoW	Preuve de Travail	Proof of Work	Algorithme de consensus utilisé dans Bitcoin
RAM	Mémoire Vive	Random Access Memory	Mémoire temporaire des serveurs
React	-	React	Bibliothèque JavaScript pour les interfaces utilisateur
REST	Transfert d'État Représentatif	Representational State Transfer	Style d'architecture pour les services web
Raft	-	Raft	Algorithme de consensus utilisé dans Hyperledger Fabric
SDK	Kit de Développement Logiciel	Software Development Kit	Ensemble d'outils pour développer des applications
SHA-256	Algorithme de Hache Sécurisé 256-bit	Secure Hash Algorithm 256-bit	Fonction de hachage cryptographique utilisée
SOC	Centre d'Opérations de Sécurité	Security Operations Center	Module de sécurité intelligent du projet
SQL	Langage de Requête Structuré	Structured Query Language	Langage pour bases de données relationnelles
TLS	Sécurité de la Couche de Transport	Transport Layer Security	Protocole de chiffrement des communications
TPS	Transactions Par Seconde	Transactions Per Second	Métrique de performance blockchain

UML	Langage de Modélisation Unifié	Unified Modeling Language	Langage de modélisation utilisé pour les diagrammes
UI/UX	Interface Utilisateur / Expérience Utilisateur	User Interface / User Experience	Conception des interfaces et de l'expérience utilisateur
VIN	Numéro d'Identification du Véhicule	Vehicle Identification Number	Identifiant unique d'un véhicule
Vite	-	Vite	Outil de développement frontend (rapide et léger)

- **Acronymes Spécifiques au Projet**

Acronyme	Signification	Contexte d'utilisation
Traffic Core	Système de Gestion de Trafic Cœur	Nom du projet principal
TrafficAuthority	Autorité du Trafic	Organisation 1 dans Hyperledger Fabric
MobilityServices	Services de Mobilité	Organisation 2 dans Hyperledger Fabric
SensorNetwork	Réseau de Capteurs	Organisation 3 dans Hyperledger Fabric
RoadID	Identifiant de Route	Identifiant unique d'une route (ex: ROAD-001)
SensorID	Identifiant de Capteur	Identifiant unique d'un capteur IoT
VehicleID	Identifiant de Véhicule	Identifiant unique d'un véhicule
DriverID	Identifiant de Conducteur	Identifiant unique d'un conducteur
ViolationID	Identifiant de Violation	Identifiant unique d'une infraction
Mistral 7B	Modèle Mistral 7 Milliards de paramètres	Modèle LLM utilisé pour l'analyse de sécurité
LM Studio	Logiciel de Gestion de Modèles de Langage	Outil pour exécuter localement des modèles LLM
CASABLANCA	Ville de Casablanca	Ville utilisée comme cas d'étude dans la simulation

Ce chapitre présente le cadre général du projet de **Smart City décentralisée** en utilisant **Hyperledger Fabric** et la blockchain. Il aborde d'abord le **contexte des Smart Cities**, mettant en évidence les défis liés à l'urbanisation, à la gestion des ressources et à l'évolution technologique des villes modernes. Il décrit ensuite les **problématiques des architectures centralisées**, notamment les points de défaillance, les risques de sécurité et les difficultés de scalabilité. Le projet proposé vise à résoudre ces problématiques en créant un **système décentralisé** pour la gestion des services urbains, en particulier le **module Traffic Core**, qui assure une gestion dynamique et transparente du trafic via la blockchain. Le chapitre se termine par la présentation des **objectifs généraux et spécifiques** du projet, détaillant les fonctionnalités et les résultats attendus, notamment en termes de **traçabilité, automatisation et sécurité**.

1.1. Contexte des Smart Cities

Les Smart Cities représentent une réponse aux défis actuels posés par l'urbanisation rapide, les changements climatiques et l'augmentation des besoins énergétiques dans les zones urbaines. Une Smart City utilise des technologies avancées, telles que l'Internet des objets (IoT), l'analyse de données massives (Big Data) et l'intelligence artificielle (IA), pour améliorer la gestion urbaine, optimiser les ressources, et rendre les villes plus durables et agréables à vivre.

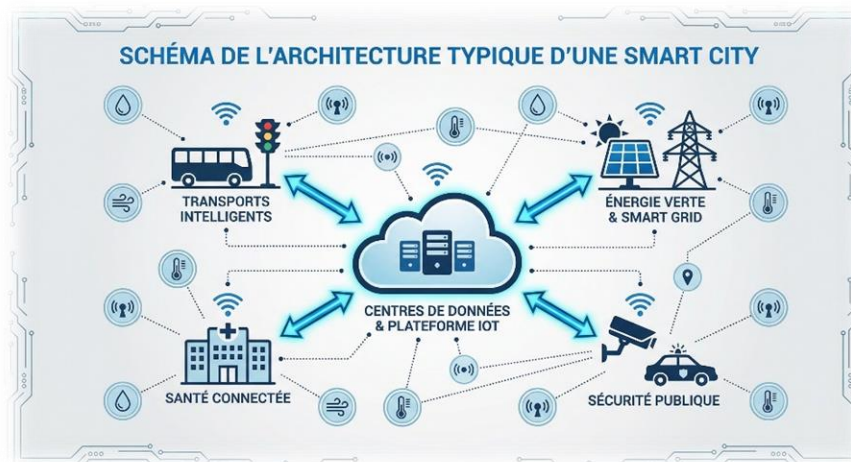


Figure 1: Schéma de l'architecture typique d'une Smart City

1.1.1. Évolution et besoins des villes modernes

Les villes connaissent une urbanisation exponentielle : selon l'ONU, en 2050, près de 70 % de la population mondiale vivra dans des zones urbaines. Cette croissance nécessite de repenser la gestion des infrastructures, des services publics, et de la mobilité pour éviter la congestion, la pollution, et l'épuisement des ressources. Une Smart City cherche à répondre à ces défis en transformant des villes traditionnelles en écosystèmes intelligents où tout est connecté et optimisé pour le bien-être des citoyens.

1.1.2. Les composantes clés des Smart Cities

Les Smart Cities reposent sur l'intégration de plusieurs technologies et systèmes de gestion :

- IoT (Internet des Objets) :
 - Des capteurs collectent des données en temps réel sur des variables essentielles telles que la qualité de l'air, la température, le trafic, et la consommation énergétique.

- Ces données sont analysées et traitées pour améliorer les services urbains (ex. ajustement des feux de circulation, gestion de l'éclairage public).
- Big Data et Analytique :
 - Traitement des données massives pour optimiser la gestion des ressources, anticiper les besoins futurs, et prendre des décisions éclairées en matière de gestion des infrastructures (par exemple, gérer la demande énergétique en fonction des prévisions météo).
- Mobilité et Transport :
 - Des solutions de mobilité intelligente (transports publics, partage de voitures, stationnement intelligent) sont mises en place pour réduire la congestion, diminuer les émissions de gaz à effet de serre, et améliorer la qualité de vie urbaine.
 - Le véhicule autonome, intégré dans les Smart Cities, peut aussi réduire les embouteillages et les accidents de la route.
- Gestion des Ressources (Eau, Énergie, Déchets) :
 - Optimisation des infrastructures grâce à des capteurs pour mesurer et réduire la consommation d'énergie et d'eau, et améliorer le recyclage des déchets. Par exemple, des capteurs dans les poubelles peuvent signaler quand elles sont pleines pour améliorer le système de collecte.
- Services Publics Connectés :
 - Les services urbains comme la sécurité publique, la gestion des urgences, et l'administration municipale peuvent être plus réactifs et efficaces grâce à des systèmes numériques interconnectés.

1.1.3. Objectifs des Smart Cities

Les Smart Cities ont plusieurs objectifs stratégiques :

- Optimisation des ressources : Réduire les coûts énergétiques, améliorer l'utilisation de l'eau, et assurer la durabilité des infrastructures.

- Réduction de l'empreinte écologique : Réduire les émissions de gaz à effet de serre (GES) et améliorer la gestion des déchets.
- Amélioration de la qualité de vie : Créer un environnement de vie plus sain, plus sûr, et plus agréable pour les citoyens, en utilisant des technologies comme la mobilité intelligente et les systèmes de surveillance.
- Renforcement de l'engagement citoyen : Utiliser des plateformes participatives pour permettre aux citoyens de contribuer à la prise de décision et à la gestion de la ville.

1.2. Problématiques des architectures centralisées

Les systèmes centralisés ont longtemps été le modèle dominant dans la gestion des services urbains, mais ils sont de plus en plus perçus comme des solutions inefficaces et vulnérables, en particulier dans le contexte des Smart Cities. Ces architectures centralisées reposent sur un point unique de contrôle (souvent un serveur ou un centre de données central) où toutes les données sont collectées, stockées et traitées. Cependant, cette approche présente plusieurs limites importantes qui rendent son application dans une Smart City peu optimale.

1.2.1. Point de défaillance unique

L'un des problèmes majeurs des architectures centralisées est qu'elles reposent sur un serveur central ou un ensemble de serveurs pour gérer toutes les données et processus. En cas de panne de ce serveur ou de compromission du système, l'ensemble de l'infrastructure peut être affecté. Par exemple, une panne de serveur pourrait entraîner l'arrêt de tous les services urbains interconnectés : gestion du trafic, éclairage public, distribution d'énergie, etc. Ce point de défaillance unique est une vulnérabilité grave dans le cadre d'une Smart City où l'objectif est de maintenir des services critiques en fonctionnement sans interruption.

1.2.2. Problèmes de sécurité et de confidentialité

Les architectures centralisées présentent également un risque accru de sécurité. Si toutes les données sont stockées dans un même endroit, elles deviennent une cible privilégiée pour les cyberattaques. Des attaques comme les attaques DDoS (Distributed Denial of Service), les injections SQL, ou le vol de données peuvent avoir des conséquences dramatiques sur la gestion des services urbains, surtout si des données sensibles, telles que les données personnelles des citoyens, sont compromises. Les données centralisées sont également plus susceptibles d'être

modifiées ou falsifiées, ce qui crée une perte de confiance de la part des citoyens dans le système.

1.2.3. Manque de transparence et de contrôle

Dans une architecture centralisée, les décisions sont souvent prises par une entité centrale, souvent dans un environnement opaque. Cela réduit la transparence et le contrôle des citoyens sur la gestion des services publics. Par exemple, les données recueillies peuvent être utilisées de manière inappropriée ou non divulguée. Dans le cadre des Smart Cities, la transparence est cruciale pour garantir que les citoyens aient confiance dans la gestion des services publics et dans l'utilisation de leurs données personnelles.

1.2.4. Manque de scalabilité

Les architectures centralisées peuvent être difficiles à scaler (étendre) à mesure que les villes croissent. La gestion des données massives générées par une Smart City (par exemple, les milliers de capteurs IoT répartis dans toute la ville) devient complexe et coûteuse dans un modèle centralisé. Il devient difficile de gérer et de traiter en temps réel une telle quantité de données tout en maintenant une haute performance et des coûts abordables. À mesure que la ville se développe, l'infrastructure centralisée doit être constamment mise à jour et élargie, ce qui entraîne des coûts élevés et une complexité accrue.

1.2.5. Problèmes de résilience et d'indépendance

En cas de défaillance d'un composant clé du système central (par exemple, le serveur central ou le réseau de communication), la résilience du système est gravement compromise. Cela peut entraîner des pannes systémiques affectant plusieurs services urbains simultanément. De plus, les architectures centralisées n'encouragent pas l'indépendance des différents services, car tous les processus sont étroitement intégrés. Par exemple, si un module de gestion du trafic rencontre une panne, il peut affecter la gestion de l'énergie ou des transports publics, ce qui empêche une gestion autonome de chaque module.

1.3. Présentation du projet de Smart City décentralisée

Le projet de Smart City décentralisée repose sur l'idée de construire une ville intelligente en utilisant une blockchain décentralisée pour interconnecter et sécuriser les différents services urbains. Le projet vise à résoudre les problèmes des systèmes centralisés en utilisant

Hyperledger Fabric, une plateforme de blockchain permissionnée qui permet de garantir la sécurité, l'intégrité, et la transparence des données.

Le but est de créer un simulateur de Smart City décentralisée où les services tels que la gestion du trafic, le stationnement, l'énergie, et les urgences sont interconnectés et fonctionnent de manière autonome tout en interagissant avec les autres modules via la blockchain.

Modules concernés par le projet :

- **Trafic** : Gestion dynamique des routes et des véhicules.
- **Stationnement** : Gestion intelligente des places de stationnement, avec réservation et paiement.
- **Énergie** : Optimisation de la consommation énergétique.
- **Urgences** : Coordination des interventions d'urgence en temps réel.

Le choix de la blockchain, en particulier d'**Hyperledger Fabric**, permet de bénéficier de la **sécurité** et de la **résilience** nécessaires pour ces services critiques, tout en garantissant une gestion transparente et auditable des transactions.

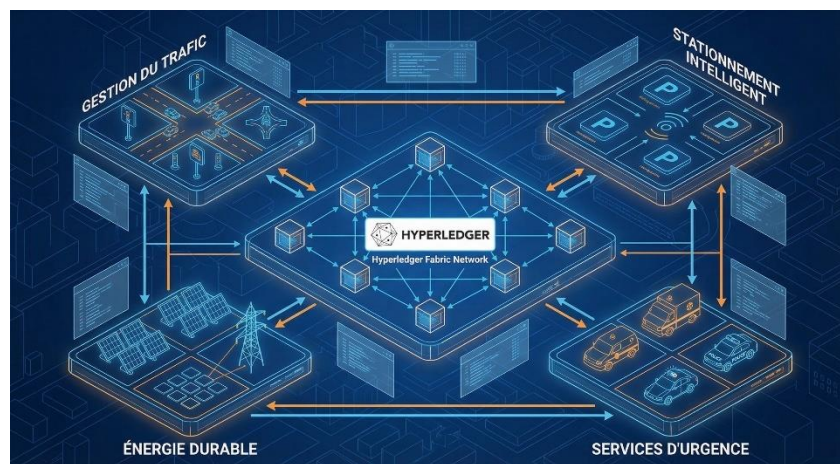


Figure 2: Schéma du projet de Smart City décentralisée avec Hyperledger Fabric

1.4. Spécifications techniques du module Traffic Core

Le module Traffic Core est le cœur du projet visant à simuler la gestion du trafic urbain dans une Smart City décentralisée. En utilisant Hyperledger Fabric pour la gestion des données en temps réel, le module garantit la traçabilité, l'intégrité et la transparence des informations relatives au trafic, tout en permettant une gestion dynamique du système.

Le Traffic Core repose sur des smart contracts déployés sur la blockchain, qui automatisent la gestion du trafic et permettent l'intégration avec d'autres services urbains.

Fonctionnalités principales du module Traffic Core :

- **Gestion du trafic en temps réel :**
 - Mise à jour continue des **statuts des routes, intersections, et véhicules**. Cette fonctionnalité garantit que les informations sont toujours à jour, permettant une gestion réactive du trafic.
- **Représentation des mouvements de véhicules :**
 - Enregistrement des entrées et sorties de véhicules dans des zones spécifiques, garantissant une **traçabilité complète** de chaque véhicule circulant dans la ville. Cela permet de suivre et d'analyser les déplacements en temps réel.
- **Ajustement dynamique des infrastructures urbaines :**
 - Le module permet d'ajuster en temps réel le **nombre de véhicules**, la **répartition des routes**, et la **gestion des croisements** selon les conditions de trafic. Cette fonctionnalité permet de fluidifier la circulation et d'optimiser les flux de trafic.
- **Mécanismes de consensus :**
 - Le module implémente et teste **deux mécanismes de consensus : PBFT (Practical Byzantine Fault Tolerance) et PoA (Proof of Authority)**. Ces mécanismes garantissent la **sécurité**, la **répartition des décisions** et la **validité des transactions** liées au trafic.
- **Optimisation des processus de gestion :**
 - Les **transactions** liées à la gestion du trafic sont automatisées par des **smart contracts**, permettant ainsi une prise de décision **automatique** en fonction des critères de gestion du trafic en temps réel (par exemple, ajuster les feux de circulation en fonction de la densité du trafic).

Tableau 1:Résumé des fonctionnalités du projet Traffic Core

Fonctionnalité	Description
Décentralisation des données	Utilisation de Hyperledger Fabric pour stocker les transactions de manière décentralisée
Gestion des routes	Modélisation des routes, intersections, et ajustement en temps réel
Gestion des violations	Détection automatique des violations de trafic (vitesse, stationnement)
Simulateur de trafic	Simulation du trafic avec la possibilité d'ajuster le nombre de véhicules et de routes
Sécurité et SOC	Intégration d'un Security Operations Center (SOC) pour la gestion des menaces

Ce module permet d'offrir un système de gestion de trafic sécurisé, efficace, et évolutif, qui peut être intégré à d'autres services urbains.

1.5. Objectifs du projet Traffic Core

Le projet Traffic Core vise à simuler et à gérer le trafic urbain d'une Smart City décentralisée en utilisant la technologie blockchain. En se basant sur Hyperledger Fabric, le projet cherche à créer un système fiable et sécurisé qui pourra interagir avec d'autres services urbains pour optimiser la gestion du trafic.

Les principaux objectifs du projet sont les suivants :

- **Gestion dynamique du trafic en temps réel :**
 - Assurer une mise à jour continue des données de trafic telles que les **routes**, **intersections**, et **véhicules** sur la blockchain. Cela permet une vue en temps réel de l'état du trafic dans toute la ville.
 - Intégrer des **smart contracts** pour automatiser la gestion du trafic en fonction de l'évolution en temps réel des conditions de circulation.
- **Traçabilité des mouvements de véhicules :**
 - Chaque mouvement de véhicule dans la ville (entrée/sortie de zones spécifiques) sera enregistré de manière sécurisée sur la blockchain, garantissant ainsi une **traçabilité complète** des déplacements et permettant d'analyser les comportements de circulation.

- **Ajustement dynamique des infrastructures urbaines :**
 - Le module **Traffic Core** doit permettre l'ajustement du nombre de véhicules circulant sur une route, la modification de la gestion des croisements et intersections, et l'ajustement des voies en fonction du trafic en temps réel.
 - Des **algorithmes d'optimisation** seront utilisés pour ajuster ces paramètres en fonction des conditions de circulation, ce qui garantit un système flexible et réactif.
- **Mise en œuvre de mécanismes de consensus :**
 - Le projet implémente et teste plusieurs mécanismes de consensus dans un environnement décentralisé pour valider les transactions liées au trafic. Les **mécanismes PBFT** (Practical Byzantine Fault Tolerance) et **PoA** (Proof of Authority) seront expérimentés afin de comparer leur efficacité en matière de sécurité, de latence et de scalabilité.
- **Automatisation des processus de gestion du trafic :**
 - Le système repose sur des **smart contracts** qui automatisent les décisions en matière de gestion du trafic (par exemple, ajuster les feux de circulation ou signaler une violation de vitesse) en fonction de critères pré-définis, sans nécessiter d'intervention humaine.

1.6. Objectifs spécifiques du projet

Les **objectifs spécifiques** du projet **Traffic Core** sont des actions précises qui vont permettre d'atteindre les objectifs généraux du projet. Ces objectifs spécifiques se concentrent sur la mise en œuvre de fonctionnalités pratiques et sur l'expérimentation de nouvelles approches technologiques pour optimiser la gestion du trafic urbain dans une **Smart City décentralisée**.

- **Conception des smart contracts pour la gestion du trafic :**
 - Développer des **smart contracts** spécifiques pour gérer les transactions liées aux routes, aux intersections et aux véhicules.

- Ces contrats permettront de garantir la **sécurisation** et la **transparence** des données sur la blockchain, tout en facilitant les échanges entre les différents modules du système.
- **Mise en place d'un simulateur de trafic intelligent :**
 - Créer un **simulateur de trafic** permettant de simuler des scénarios urbains (par exemple, l'ajout de nouveaux véhicules ou la création de nouvelles routes).
 - Ce simulateur aidera à tester l'impact des décisions prises par les **smart contracts** en temps réel, et à ajuster les paramètres pour obtenir des résultats optimaux.
- **Expérimentation et comparaison des mécanismes de consensus :**
 - Tester et comparer la performance de **PBFT** et **PoA** dans la gestion du trafic urbain.
 - Ces tests incluront des évaluations de la **latence**, du **throughput**, et de la **scalabilité** des deux mécanismes dans le contexte d'une Smart City décentralisée.
- **Intégration avec d'autres modules de la Smart City :**
 - Intégrer **Traffic Core** avec d'autres modules de la Smart City, comme la gestion de l'énergie, du stationnement, et des urgences, pour tester la communication et l'interopérabilité des services via la blockchain.
 - Par exemple, si un véhicule est en infraction, cela peut déclencher une action dans le module **stationnement** ou **urgences**.
- **Détection et gestion des violations de trafic en temps réel :**
 - Implémenter un système permettant la **détection automatique** des violations de trafic (par exemple, excès de vitesse, stationnement illégal) et la gestion des **amendes** en temps réel.
 - Cette fonctionnalité permet d'automatiser la gestion des violations sans intervention humaine, tout en garantissant une transparence totale via la blockchain.

- **Création d'une interface utilisateur pour le contrôle et la visualisation :**
 - Développer une interface utilisateur (via un **tableau de bord**) permettant de visualiser en temps réel les **données de trafic**, les **violations**, et les **statistiques** associées.
 - Cette interface permettra aux utilisateurs de suivre l'état des routes, d'interagir avec le système et d'analyser les résultats des simulations de trafic.

2. Fondements Théoriques et État de l'Art

Ce chapitre explore les fondements théoriques et l'état de l'art concernant les technologies sous-jacentes du projet de **Smart City décentralisée**, en mettant l'accent sur la **blockchain**, en particulier **Hyperledger Fabric**, et son utilisation dans la gestion des services urbains. Il se divise en plusieurs sections qui couvrent l'**introduction à la blockchain**, l'**écosystème Hyperledger**, les **smart contracts**, ainsi que l'intégration des **Smart Cities** avec ces technologies.

2.1. Introduction à la blockchain

La **blockchain** est une technologie décentralisée permettant de stocker et de partager des données de manière sécurisée, transparente et immuable. Elle fonctionne comme un **registre distribué** où chaque transaction est enregistrée sous forme de **bloc** et liée à un bloc précédent, formant ainsi une chaîne continue et vérifiable.

La blockchain repose sur des concepts de **cryptographie** et de **consensus** pour garantir l'intégrité des données et la fiabilité des transactions sans nécessiter d'autorité centrale. Cette technologie a gagné en popularité grâce à son utilisation dans les **cryptomonnaies**, mais elle présente également un potentiel énorme pour d'autres domaines, notamment la gestion des services urbains dans les **Smart Cities**.

2.1.1. Fonctionnement de la Blockchain

La blockchain fonctionne selon un mécanisme où chaque **bloc** contient une **liste de transactions** validées par les participants du réseau. Une fois validé, un bloc est ajouté à la chaîne, rendant ainsi toute modification ultérieure pratiquement impossible sans que cela ne soit détecté.

Les trois composants clés de la blockchain sont :

- **Les nœuds** : Chaque participant du réseau.
- **Le consensus** : Le mécanisme par lequel les nœuds valident les transactions.
- **Les smart contracts** : Des programmes automatisés exécutés sur la blockchain pour valider les transactions et automatiser des processus.

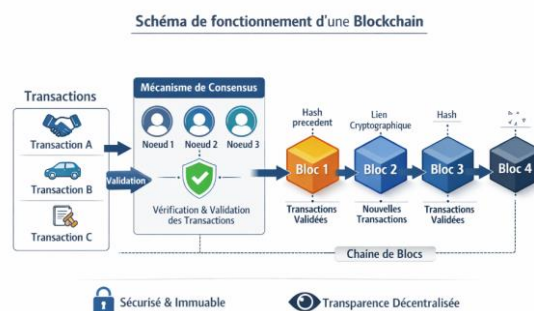


Figure 3: Schéma de fonctionnement d'une blockchain

2.1.2. Types de Blockchain

Il existe plusieurs types de blockchains, chacun ayant des caractéristiques uniques qui le rendent adapté à des cas d'usage spécifiques. Les deux principaux types de blockchain sont :

- **Blockchain publique**

Une blockchain publique est totalement décentralisée et ouverte à tous. Elle ne nécessite aucune autorisation pour rejoindre le réseau, et tout participant peut vérifier, valider et ajouter des transactions à la blockchain. Ce modèle repose sur la confiance distribuée entre les participants du réseau, et son objectif principal est de maintenir l'intégrité et la transparence des transactions.

Caractéristiques de la Blockchain publique :

- **Décentralisation totale** : Il n'y a pas d'autorité centrale contrôlant la blockchain. Tous les participants sont égaux et contribuent au fonctionnement du réseau.
- **Accessibilité ouverte** : N'importe qui peut rejoindre le réseau, valider des transactions et ajouter des blocs, sans restriction.
- **Transparence** : Toutes les transactions sont visibles par tous les participants, et chaque bloc est immuable une fois ajouté à la chaîne.
- **Sécurité par consensus** : Les mécanismes de consensus (comme **Proof of Work (PoW)** pour Bitcoin) sont utilisés pour valider les transactions, garantir la sécurité du réseau, et empêcher les attaques malveillantes.

- **Blockchain privée/permissionnée**

Une blockchain privée (également appelée blockchain permissionnée) est un réseau blockchain dans lequel l'accès et la participation sont restreints. Seuls les participants autorisés peuvent rejoindre le réseau, valider des transactions ou accéder aux données. Ce type de blockchain est souvent utilisé par des entreprises ou des consortiums qui ont besoin de contrôler les parties prenantes et la confidentialité des transactions.

Caractéristiques de la Blockchain privée/permissionnée :

- **Accès restreint** : L'adhésion au réseau est limitée à un groupe spécifique de participants autorisés, ce qui permet de mieux contrôler qui peut valider les transactions et modifier l'état de la blockchain.

- **Contrôle centralisé (à un certain degré)** : Bien que décentralisée dans son fonctionnement, le contrôle de la blockchain peut être attribué à un nombre restreint d'entités, comme une organisation ou un consortium.
- **Mécanismes de consensus adaptés** : Dans les blockchains privées, des mécanismes de consensus plus efficaces (comme **PBFT** (Practical Byzantine Fault Tolerance) ou **Proof of Authority (PoA)**) peuvent être utilisés, ce qui permet des transactions plus rapides et plus économiques.
- **Confidentialité** : Les transactions peuvent être privées ou visibles uniquement pour certains participants, ce qui est crucial dans des environnements sensibles.

2.2. Hyperledger Fabric : Vue d'ensemble

Hyperledger Fabric est une plateforme de blockchain permissionnée qui permet de développer des applications de blockchain à la fois sécurisées et évolutives. Contrairement à des blockchains publiques comme Bitcoin, Hyperledger Fabric est conçu pour des environnements où les participants doivent être identifiés et autorisés.

2.2.1. Architecture d'Hyperledger Fabric

Hyperledger Fabric repose sur une architecture modulaire qui sépare les tâches de gestion des transactions, de consensus, et de validation des données. Voici les composants clés :

- **Les pairs** : Ils exécutent les smart contracts et valident les transactions.
- **Les orderers** : Ils assurent l'ordre des transactions dans le réseau.
- **Les canaux** : Ils permettent de créer des sous-réseaux privés pour des transactions entre un sous-ensemble de participants.
- **Les chaincodes** : Ce sont des smart contracts qui gèrent la logique transactionnelle dans le réseau.

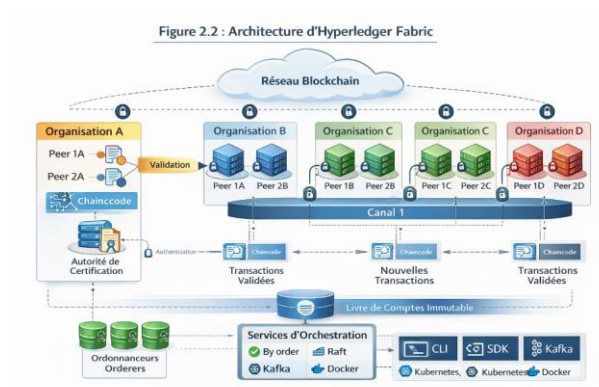


Figure 4: Architecture d'Hyperledger Fabric

2.3. Les Smart Contracts et Chaincode dans Fabric

Les smart contracts sont des programmes autonomes qui s'exécutent automatiquement lorsque des conditions préalables sont remplies. Dans Hyperledger Fabric, les smart contracts sont appelés chaincodes. Ils sont écrits dans des langages comme Go, JavaScript, ou Java et permettent d'implémenter des règles d'affaires.

2.3.1. Fonctionnement des Smart Contracts

Les smart contracts, ou chaincodes, permettent de définir des règles d'interaction entre les participants du réseau. Ils peuvent valider des transactions, exécuter des calculs ou modifier l'état des données stockées sur la blockchain, sans nécessiter d'intervention humaine.

2.4. Les Smart Cities et l'intégration des technologies blockchain

Les Smart Cities utilisent une combinaison de technologies avancées, telles que l'IoT, les Big Data, et la blockchain, pour améliorer la gestion des services urbains. La blockchain peut être utilisée pour gérer et sécuriser des services comme le trafic, le stationnement, l'énergie, et bien d'autres aspects des infrastructures urbaines.

2.4.1. Applications de la Blockchain dans les Smart Cities

Les principales applications de la **blockchain** dans les **Smart Cities** incluent :

- **Gestion du trafic** : Enregistrer et valider les mouvements des véhicules et des données de circulation en temps réel.

- **Stationnement intelligent** : Utiliser la blockchain pour la gestion des places de stationnement et le paiement automatique.
- **Énergie** : Optimiser la gestion de la consommation d'énergie et intégrer des systèmes de **micro-réseaux** pour la production et la distribution d'énergie renouvelable.

Tableau 2: Exemples d'applications de la blockchain dans les Smart Cities

Application	Description
Gestion du trafic	Suivi et gestion des flux de véhicules en temps réel sur la blockchain
Stationnement	Systèmes de gestion des places de stationnement, réservation, et paiement
Énergie	Optimisation des réseaux d'énergie et gestion des données de consommation
Sécurité publique	Utilisation de la blockchain pour la gestion des incidents de sécurité

2.5. Mécanismes de Consensus dans Hyperledger Fabric

Les mécanismes de consensus sont essentiels pour valider les transactions dans un réseau blockchain. Dans Hyperledger Fabric, plusieurs mécanismes de consensus sont disponibles, permettant de répondre aux besoins spécifiques de chaque réseau.

2.5.1. Types de consensus dans Hyperledger Fabric

Dans Hyperledger Fabric, le mécanisme de consensus est un élément clé pour garantir la validité des transactions et assurer l'intégrité du réseau. Contrairement aux blockchains publiques qui utilisent des mécanismes comme Proof of Work (PoW) ou Proof of Stake (PoS), Hyperledger Fabric repose sur des algorithmes de consensus adaptés aux environnements privés et permissionnés, permettant des transactions plus rapides et plus sécurisées.

Hyperledger Fabric offre plusieurs mécanismes de consensus, parmi lesquels les plus utilisés sont Raft, PBFT (Practical Byzantine Fault Tolerance) et Kafka. Chacun de ces mécanismes présente des avantages et des inconvénients, et le choix du mécanisme dépend des besoins spécifiques du réseau.

1. Raft

Raft est un algorithme de consensus utilisé dans Hyperledger Fabric pour assurer la cohérence des données entre les nœuds du réseau. Raft est conçu pour être plus simple et plus performant

que les autres algorithmes de consensus, tout en maintenant un haut niveau de sécurité. Il est basé sur un système de **leadership** où un nœud est désigné comme **leader**, et les autres nœuds sont des **followers**.

- **Principe de fonctionnement :**

- Un nœud leader est responsable de l'ordre des transactions et de leur propagation dans le réseau.
- Les nœuds **followers** répliquent les transactions reçues du leader.
- Si le leader échoue ou devient inactif, un nouveau leader est élu parmi les nœuds actifs, ce qui garantit une continuité du service.

2. PBFT (Practical Byzantine Fault Tolerance)

PBFT est un algorithme de consensus qui garantit la sécurité même en présence de nœuds malveillants (jusqu'à un tiers des nœuds peuvent être compromis sans compromettre l'intégrité du réseau). PBFT est souvent utilisé dans des environnements où la sécurité est primordiale, comme les applications bancaires ou gouvernementales.

- **Principe de fonctionnement :**

- Chaque transaction est validée par un **nombre de nœuds** plutôt que par un seul leader, et les transactions sont ajoutées au **registre** uniquement si un consensus est atteint entre les nœuds.
- **Trois étapes principales :**
 1. **Pré-vote** : Les nœuds valident la transaction en émettant un vote.
 2. **Vote** : Si un certain nombre de nœuds (généralement plus de 2/3) votent en faveur d'une transaction, elle est considérée comme valide.
 3. **Committing** : Une fois les votes validés, la transaction est commitée et ajoutée à la chaîne de blocs.

3. Kafka

Kafka est un autre algorithme de consensus utilisé dans Hyperledger Fabric, principalement pour les réseaux qui nécessitent une forte **performance** et une **gestion d'événements en temps**

réel. Kafka est basé sur un système de journalisation distribué, où les messages (ou transactions) sont traités dans un ordre spécifique.

- **Principe de fonctionnement :**

- Kafka repose sur la notion de **brokers**, qui stockent les messages dans des **partitions**.
- Les transactions sont envoyées aux **partitions** et sont ensuite validées et ordonnées par les nœuds du réseau. Les **brokers** garantissent que l'ordre des transactions est respecté.
- Contrairement à Raft ou PBFT, Kafka ne garantit pas une tolérance complète aux fautes Byzantines, mais il est efficace pour les réseaux où la performance est plus importante que la résilience complète face aux fautes.

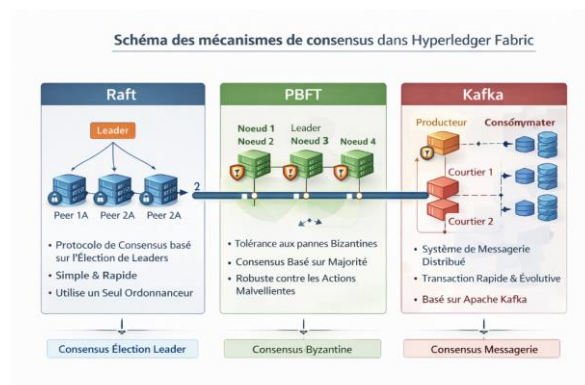


Figure 5: Schéma des mécanismes de consensus dans Hyperledger Fabric (Raft, PBFT, etc.)

3. Conception du Système Traffic Core

Ce chapitre présente la conception détaillée du **système Traffic Core**, qui constitue le cœur du projet de **Smart City décentralisée**. Il explique comment les **smart contracts**, la **blockchain** et les autres technologies sont utilisées pour créer un système de gestion du trafic dynamique, sécurisé et évolutif, adapté aux besoins d'une **Smart City**.

3.1. Vue d'ensemble du module Traffic Core

Le module Traffic Core est l'élément central du système de gestion du trafic dans une Smart City décentralisée. En utilisant Hyperledger Fabric, il assure la sécurité, la transparence et la traçabilité des données relatives au trafic. Ce module permet de gérer et simuler en temps réel le trafic urbain, en optimisant l'utilisation des infrastructures, en détectant les violations de trafic et en ajustant dynamiquement les paramètres du système en fonction des conditions.

Fonctionnalités principales du module Traffic Core :

- **Gestion du trafic en temps réel** : Surveillance et mise à jour des conditions de circulation.
- **Suivi des mouvements de véhicules** : Traçabilité complète des véhicules circulant dans la ville.
- **Ajustement dynamique** des infrastructures et de la gestion des feux de circulation.
- **Détection des violations** : Suivi des infractions telles que les excès de vitesse ou le stationnement illégal.

3.2. Architecture réseau Hyperledger Fabric

Dans un Traffic core , le réseau Hyperledger Fabric est configuré avec trois organisations principales, chacune ayant un rôle spécifique dans la gestion du trafic. Chaque organisation dispose de pairs (peers) qui exécutent des smart contracts (chaincodes) et valident les transactions dans le système. Les orderers sont responsables de l'organisation des transactions et de leur ajout à la blockchain.

Voici un aperçu détaillé des organisations et de leurs responsabilités respectives dans le système de gestion du trafic :

1. TrafficAuthority (Organisation 1 - TrafficAuthorityOrg1)

- MSP ID : TrafficAuthorityOrg1MSP
- Nombre de Peers : 2 (7051, 7061)
- Rôle principal : Régulateur public
- Responsabilités :

- Gestion des routes et intersections : Cette organisation est responsable de la gestion des routes, des intersections, et de l'état de la circulation.
- Règles de circulation : Elle définit les règles de circulation, telles que les limites de vitesse et les priorités de passage.
- Événements de trafic : Elle enregistre et gère les événements de trafic comme les accidents et les fermetures de routes.
- Maintenance des infrastructures : Elle s'occupe de la maintenance des infrastructures routières.
- Cas d'usage typiques :
 - Créer/fermer une route : Créer une nouvelle route ou fermer une route existante en fonction des besoins de la ville.
 - Signaler un accident : Lors d'un accident, l'organisation peut enregistrer l'événement et lancer des actions pour signaler l'incident.
 - Planifier des travaux : Organisation des travaux sur les routes, comme des réparations ou des constructions.

2. MobilityServices (Organisation 2 - MobilityServicesOrg2)

- MSP ID : MobilityServicesOrg2MSP
- Nombre de Peers : 2 (8051, 8061)
- Rôle principal : Opérateur de la mobilité
- Responsabilités :
 - Registre des véhicules : Cette organisation gère les informations relatives aux véhicules circulant dans la ville.
 - Gestion des conducteurs : Elle est responsable de l'enregistrement des conducteurs et de la gestion des permis de conduire.
 - Traitement des violations : Elle prend en charge la gestion des violations de trafic, telles que les excès de vitesse et le stationnement illégal.

- Transferts de propriété : Elle gère les transactions liées au transfert de la propriété des véhicules.
- Cas d'usage typiques :
 - Immatriculer un véhicule : Lorsqu'un véhicule est nouvellement enregistré, MobilityServices peut créer une entrée pour ce véhicule dans le système.
 - Émettre une amende : Si un conducteur commet une infraction, l'organisation émet une amende et la transmet au conducteur concerné.
 - Suspendre un permis : Si un conducteur enfreint les règles, MobilityServices peut suspendre son permis.

3. SensorNetwork (Organisation 3 - SensorNetworkOrg3)

- MSP ID : SensorNetworkOrg3MSP
- Nombre de Peers : 2 (9051, 9061)
- Rôle principal : Réseau IoT (Internet des objets)
- Responsabilités :
 - Capteurs de trafic : Cette organisation gère les capteurs IoT qui mesurent en temps réel l'occupation des routes, la vitesse des véhicules et les conditions de circulation.
 - Lectures en temps réel : Les capteurs envoient des données en temps réel sur la circulation et les conditions de la route.
 - Alertes automatiques : En cas d'incidents ou d'anomalies, l'organisation envoie des alertes automatiques.
 - Qualité de l'air : En plus des données sur le trafic, SensorNetwork peut également mesurer la qualité de l'air, ce qui est important pour la gestion de l'environnement urbain.
- Cas d'usage typiques :

- Enregistrer un capteur : Ajouter un capteur IoT pour mesurer les conditions de circulation ou surveiller des zones spécifiques.
- Recevoir lecture vitesse : Enregistrer la vitesse des véhicules circulant sur les routes surveillées.
- Détecter la pollution : Les capteurs peuvent également mesurer la qualité de l'air pour évaluer l'impact environnemental du trafic.

Tableau 3: Description des organisations : TrafficAuthority, MobilityServices, SensorNetwork

Organisation	Rôle
TrafficAuthority	Responsable de la gestion des routes, des intersections, et des feux de circulation.
MobilityServices	Gère les véhicules, la gestion des voies et l'optimisation du trafic.
SensorNetwork	Supervise les capteurs IoT pour collecter des données sur la circulation et l'état du trafic.

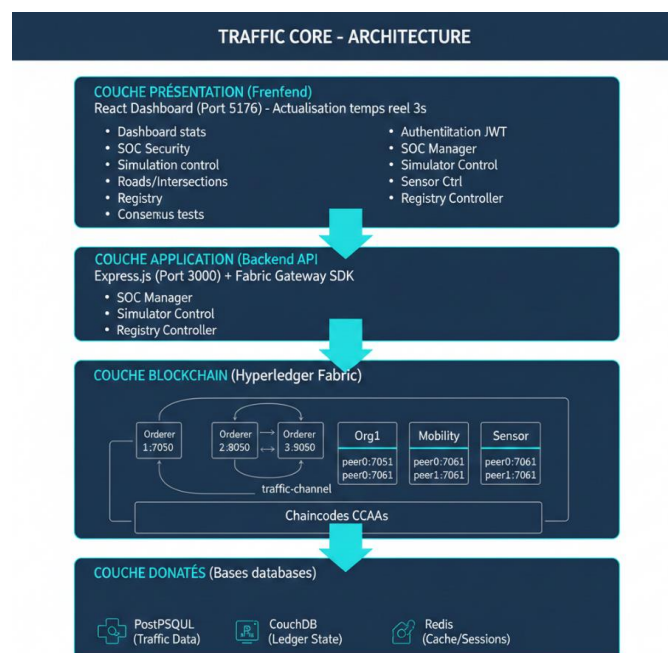


Figure 6 : Architecture réseau Traffic

3.3. Modélisation des données du trafic et IoT

Dans **Traffic Core**, le modèle de données est essentiel pour garantir l'intégration fluide entre la gestion du trafic, les capteurs IoT, et la blockchain Hyperledger Fabric. Cette approche permet de collecter, traiter, et stocker des données provenant de plusieurs sources, tout en garantissant leur traçabilité et leur sécurité via la blockchain.

3.3.1. Données des routes et intersections

Les données des routes et des intersections sont représentées sous forme d'assets dans la blockchain. Chaque route a des attributs tels que son statut (ouverte, fermée, en maintenance, etc.) et son type (autoroute, route principale, etc.). Les intersections, quant à elles, sont modélisées avec leurs configurations de feux de circulation et leur gestion du flux de trafic.

Tableau 4: Modèle des routes et intersections

Attribut	Description
RouteID	Identifiant unique de la route
Statut	État de la route (ouverte, fermée, congestionnée)
IntersectionID	Identifiant de l'intersection
Feu	Paramètres des feux de circulation

3.3.2. Capteurs IoT

Les capteurs IoT sont utilisés pour collecter des données en temps réel sur divers paramètres tels que la vitesse des véhicules, la qualité de l'air, et les conditions météorologiques. Ces données sont transmises aux nœuds de la blockchain pour être traitées et stockées. Chaque capteur est représenté comme un asset dans le réseau, avec des informations sur son emplacement, son type, et ses lectures en temps réel.

Tableau 5: Modèle des capteurs IoT

Attribut	Description
CapteurID	Identifiant unique du capteur
Type	Type de capteur (vitesse, qualité de l'air, etc.)
Emplacement	Localisation du capteur
Lecture	Valeur mesurée par le capteur

3.3.3. Véhicules et conducteurs

Les données relatives aux véhicules et conducteurs sont essentielles pour la gestion du trafic, le suivi des violations, et l'attribution des amendes. Chaque véhicule est enregistré avec son immatriculation, son statut (actif, suspendu), et les informations du conducteur associées, telles que son permis de conduire et son historique de violations.

Tableau 6: Modèle des véhicules et conducteurs

Attribut	Description
VehiculeID	Identifiant unique du véhicule
Immatriculation	Numéro d'immatriculation du véhicule
ConducteurID	Identifiant du conducteur
Statut	Statut du véhicule (actif, suspendu)

3.3.4. Relations entre les entités

Les relations entre les routes, les capteurs, les véhicules et les conducteurs sont gérées via des smart contracts. Ces contrats assurent l'intégrité des transactions et permettent d'enregistrer des événements tels que les violations de trafic (excès de vitesse, stationnement illégal), qui sont ensuite traités automatiquement pour l'application des amendes et des sanctions.

3.3.5. Intégration avec Hyperledger Fabric

Toutes ces données sont intégrées dans le réseau **Hyperledger Fabric**, avec une couche de validation et de stockage des transactions. Les **chaincodes** sont utilisés pour définir les règles métier liées à chaque entité, telles que l'enregistrement des violations ou la gestion des statuts des routes.

Chaque organisation (TrafficAuthority, MobilityServices, SensorNetwork) a son propre ensemble de **peers** qui valident les transactions avant de les inscrire dans le ledger. Les données sont ainsi partagées et mises à jour en temps réel, garantissant la transparence et la sécurité des informations.

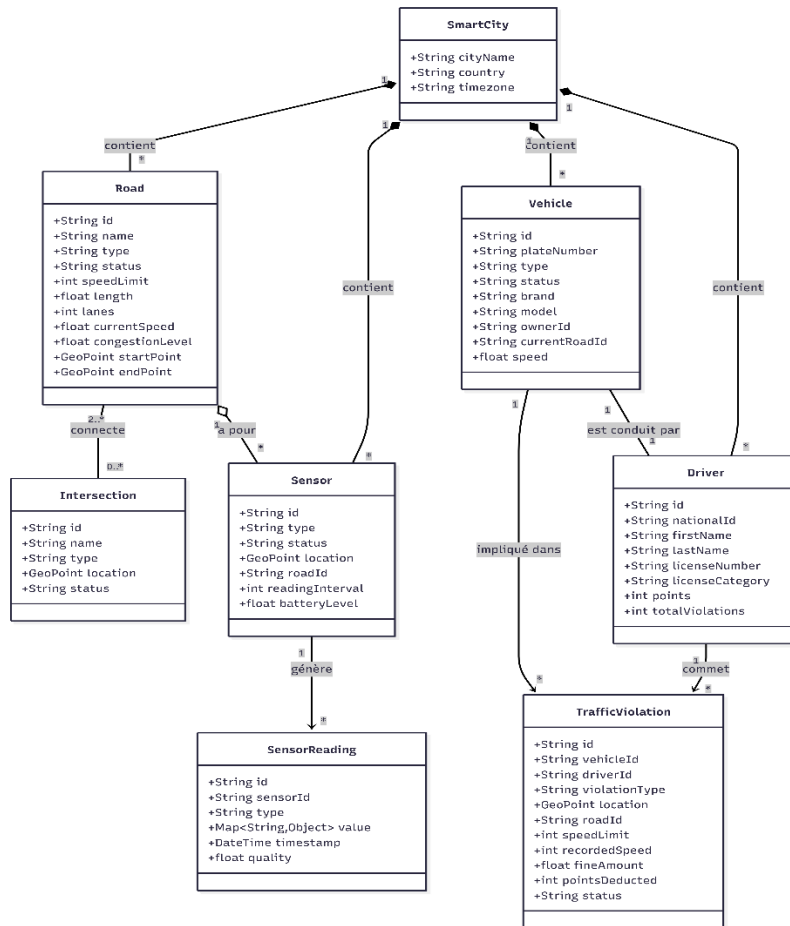


Figure 7: Diagramme UML des entités route, capteur, véhicule, conducteur

3.4. Conception des smart contracts

Les smart contracts (ou chaincodes) sont au cœur de la gestion du trafic dans ce projet de Smart City décentralisée. Ces contrats intelligents sont utilisés pour automatiser les processus liés à la gestion des routes, des véhicules, et des violations de trafic. En utilisant Hyperledger Fabric, les smart contracts garantissent la sécurité, l'intégrité, et la transparence des transactions dans le système.

Principaux Smart Contracts utilisés

Voici les principaux **chaincodes** utilisés pour gérer les diverses facettes du trafic urbain :

- **road-manager** : Ce smart contract est responsable de la gestion des **routes** et des **intersections**. Il permet de créer de nouvelles routes, de les mettre à jour ou de les supprimer, et de suivre l'état des voies et des intersections (ouverte, fermée, en maintenance).

Fonctions principales :

- CreateRoad
- UpdateRoadStatus
- GetRoad
- GetAllRoads
- **sensor-data** : Ce contract gère l'enregistrement et la mise à jour des données des **capteurs IoT**. Il permet de récupérer les données de circulation en temps réel, comme la **vitesse des véhicules** ou la **qualité de l'air**.

Fonctions principales :

- RegisterSensor
- RecordReading
- GetSensorData
- **traffic-registry** : Ce contract est responsable de l'enregistrement des **véhicules** et des **conducteurs**, ainsi que de la gestion des **violations de trafic** (par exemple, excès de vitesse). Il permet aussi de gérer les **amendes** et les **transferts de propriété** des véhicules.

Fonctions principales :

- RegisterVehicle
- RecordViolation
- TransferVehicle

Flux des transactions entre les smart contracts

Les transactions sont déclenchées en réponse à des événements externes (par exemple, la détection d'une violation par un capteur). Voici un exemple de flux :

1. Un **capteur IoT** détecte une violation (excès de vitesse).
2. Le **smart contract sensor-data** enregistre cette donnée.

3. Le **smart contract traffic-registry** valide l'infraction et crée une amende pour le conducteur.
4. Le smart contract enregistre toutes les transactions sur la **blockchain** Hyperledger Fabric pour garantir leur transparence et leur intégrité.

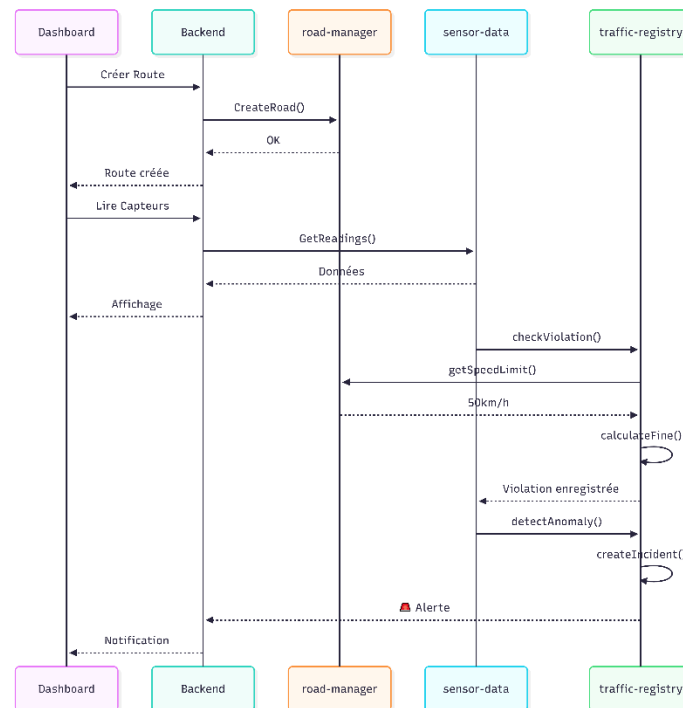


Figure 8: Diagramme des interactions entre les smart contracts

3.5. Architecture de l'application simulateur / tableau de bord

L'application simulateur / tableau de bord joue un rôle clé dans la gestion du trafic de la Smart City décentralisée. Elle permet d'afficher en temps réel les données du trafic, d'interagir avec le système, de visualiser les statistiques, et d'ajuster les paramètres du trafic pour effectuer des simulations et tester différents scénarios. Cette architecture a été développée pour fournir une interface intuitive et réactive, facilitant l'interaction avec les smart contracts et les données provenant de capteurs IoT.

Composants principaux de l'architecture du tableau de bord

L'architecture de l'application se divise en plusieurs couches, permettant une séparation claire des responsabilités entre le **frontend** (interface utilisateur) et le **backend** (logique métier et communication avec la blockchain).

1. Couche Présentation (Frontend)

La couche présentation (ou frontend) est responsable de l'interface utilisateur qui permet aux utilisateurs (gestionnaires de la ville, opérateurs, etc.) d'interagir avec le système. Elle est construite avec React et se met à jour en temps réel, fournissant des informations précises et dynamiques sur l'état du trafic, les routes, et les violations.

Principales fonctionnalités de l'interface utilisateur :

- Visualisation en temps réel des conditions de circulation : Le tableau de bord affiche les routes, les intersections, et les véhicules en circulation avec des données à jour.
- Contrôle des simulations : L'utilisateur peut ajuster les paramètres de la simulation (par exemple, ajouter des véhicules, fermer une route, ou ajuster les feux de circulation) pour observer l'impact de ces ajustements sur le trafic.
- Statistiques de trafic : Des graphiques et des tableaux montrent des statistiques en temps réel sur la vitesse moyenne, la densité du trafic, et le nombre d'infractions enregistrées.
- Gestion des alertes : Le tableau de bord permet de visualiser les alertes générées par les capteurs IoT (par exemple, pollution élevée ou congestion).
- Affichage des violations : Les infractions, telles que les excès de vitesse ou le stationnement illégal, sont automatiquement affichées avec des options pour gérer les amendes et les sanctions.

2. Couche Application (Backend API)

La couche application est responsable de la logique métier et de la communication entre le frontend et le réseau Hyperledger Fabric. Elle utilise Express.js pour gérer les demandes HTTP et communique avec la blockchain via le Fabric Gateway SDK. Cette couche est essentielle pour l'automatisation des processus liés à la gestion du trafic, comme l'ajustement des feux de circulation ou l'enregistrement des violations.

Fonctionnalités principales :

- Authentification JWT : Pour sécuriser les accès, l'API backend implémente un système d'authentification basé sur les JSON Web Tokens (JWT).
- Contrôle du simulateur : L'API backend reçoit les commandes de simulation et ajuste les paramètres du trafic, comme les feux de circulation ou les données des capteurs.

- Gestion des données des routes et véhicules : L'API backend permet de gérer et d'interroger les données relatives aux routes, intersections, véhicules et capteurs IoT dans le réseau Hyperledger Fabric.
- Interaction avec la blockchain : L'API interagit avec les smart contracts sur la blockchain pour créer des transactions liées aux ajustements du trafic, comme la modification des feux de circulation ou la mise à jour des données de circulation.

3. Couche Blockchain (Hyperledger Fabric)

La couche blockchain constitue la base de données décentralisée et immuable qui stocke toutes les transactions liées à la gestion du trafic. Elle est construite sur Hyperledger Fabric, une blockchain permissionnée qui garantit la sécurité, la transparence et la traçabilité des données.

Composants principaux :

- Peers : Les peers exécutent les smart contracts (chaincodes) et valident les transactions. Ils sont responsables du stockage des données sur la blockchain.
- Orderers : Les orderers organisent les transactions et les ajoutent à la blockchain.
- Chaincodes (Smart Contracts) : Les chaincodes définissent les règles de gestion du trafic, comme la création de routes, la gestion des infractions, et l'ajustement des feux de circulation.
- Canaux : Le réseau Hyperledger Fabric est partitionné en canaux, permettant des transactions privées entre les différentes organisations (par exemple, TrafficAuthority, MobilityServices, SensorNetwork).

Flux des interactions entre les couches

Voici comment les couches interagissent entre elles pour garantir une gestion fluide du trafic :

1. **Frontend (React)** : L'utilisateur interagit avec l'interface utilisateur pour obtenir des informations sur l'état du trafic ou ajuster les paramètres de simulation.
2. **Backend (Express.js)** : L'API backend reçoit ces commandes et les transmet aux **smart contracts** sur la blockchain **Hyperledger Fabric** via le **Fabric Gateway SDK**.

3. **Blockchain (Hyperledger Fabric) :** Les **smart contracts** exécutent la logique métier, comme l'ajustement des feux de circulation, l'enregistrement des violations de trafic, ou la mise à jour des données de la route, puis valident et enregistrent les transactions sur la blockchain.
4. **Mise à jour en temps réel :** Dès qu'une transaction est validée par la blockchain, les **peers** mettent à jour les données, et l'interface utilisateur est actualisée en temps réel pour refléter les changements.

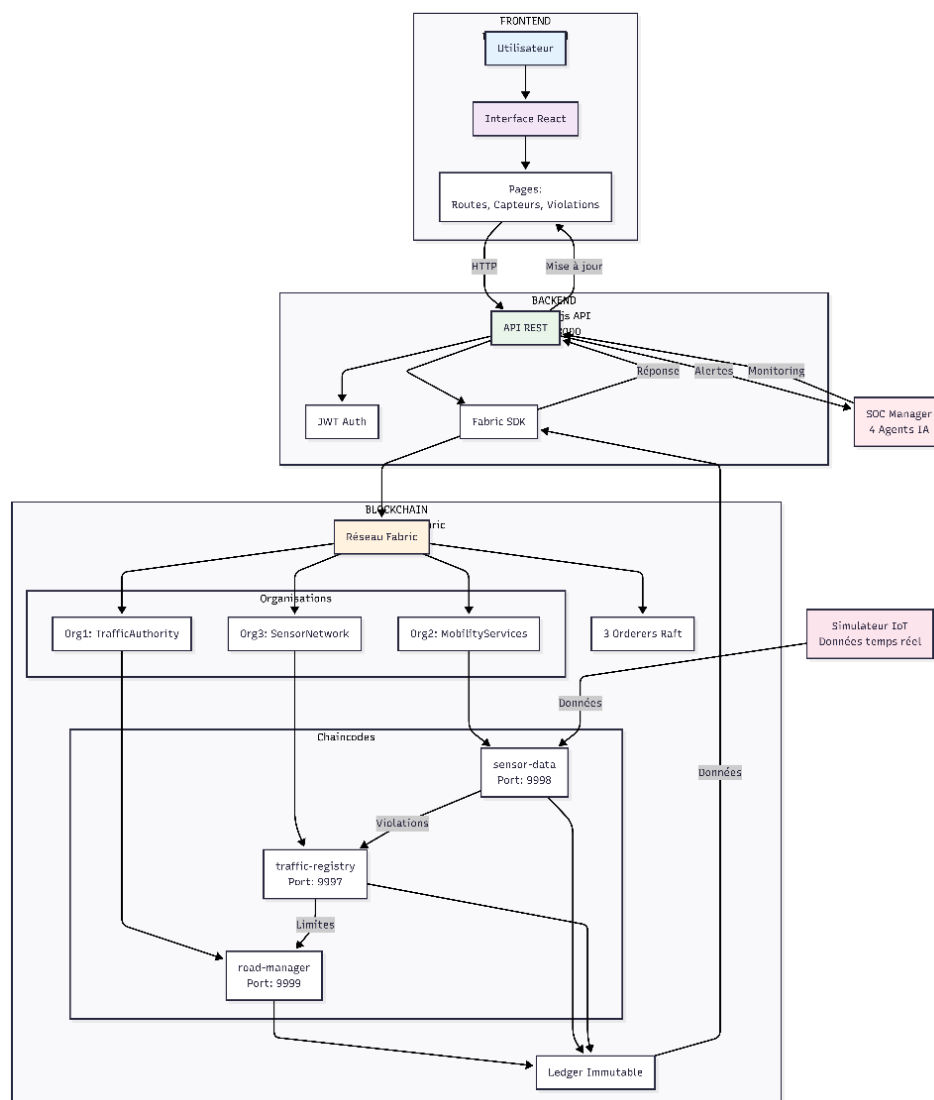


Figure 9: Schéma de l'architecture de l'application simulateur / tableau de bord

4. Implémentation et Déploiement de la Plateforme

Ce chapitre décrit le processus d'implémentation de la plateforme de gestion de trafic urbain décentralisée en utilisant Hyperledger Fabric, ainsi que les étapes de déploiement des composants essentiels comme les chaincodes, le simulateur et le tableau de bord.

4.1. Environnement technique

L'environnement utilisé pour ce projet repose sur plusieurs technologies open-source telles que **Hyperledger Fabric** pour la gestion de la blockchain, **Node.js** et **Express** pour le backend API, et **React** pour le frontend. L'ensemble du système est conteneurisé à l'aide de **Docker** pour faciliter le déploiement et l'orchestration des différents composants.

4.2. Scripts d'installation et d'orchestration

Pour simplifier le processus d'installation, plusieurs scripts ont été développés. Ces scripts permettent de configurer l'environnement, d'installer les prérequis nécessaires et de déployer la plateforme de manière automatisée.

4.2.1. Scripts d'installation (préparer l'environnement)

Ces scripts servent à installer les dépendances et outils nécessaires avant de lancer la plateforme.

- **scripts/install-prerequisites.sh**
 - **Rôle** : installer les prérequis système (Docker, Go, Node.js, etc.).
 - **Quand l'utiliser** : au tout début, sur une machine neuve.
 - **Commande** :
 - `chmod +x scripts/*.sh`
 - `./scripts/install-prerequisites.sh`
- **scripts/install-fabric.sh**
 - **Rôle** : installer Hyperledger Fabric (dans ton guide : Fabric 2.5.4).
 - **Commande** :
 - `./scripts/install-fabric.sh`

4.2.2. Scripts d'orchestration (démarrer/arrêter tout le projet)

Ces scripts automatisent le démarrage complet : réseau Fabric + chaincodes + backend + frontend + simulateur.

- **start-all.sh**

- **Rôle** : lancement complet du projet.
- **Ce que ça démarre (selon ton guide) :**
 - Réseau Hyperledger Fabric
 - 3 chaincodes (**road-manager**, **sensor-data**, **traffic-registry**)
 - Backend API (port 3000)
 - Frontend Dashboard (port 5176)
 - Simulateur IoT
- **Commande** : `./start-all.sh`

- **stop-all.sh**

- **Rôle** : arrêt complet de la plateforme (réseau + services).
- **Commande** : `./stop-all.sh`

4.2.3. Script d'orchestration du réseau blockchain (Fabric)

Ce script est celui qui **monte/descend le réseau Fabric** et **déploie les chaincodes**.

- **scripts/network.sh**

- **Rôle** : gérer le cycle de vie du réseau Fabric.
- **Commandes utilisées dans ton guide :**
 - `./scripts/network.sh up` → démarrer le réseau blockchain
 - `./scripts/network.sh deployCC` → déployer les chaincodes
 - (et généralement aussi `down` pour arrêter, même si tu l'expliques plus loin)
- **À noter** : dans ta structure projet, ce script est explicitement décrit comme "*up, down, deployCC*".

- **Script interne (helper)**
 - **network/scripts/envVar.sh**
 - **Rôle** : script utilitaire pour variables d'environnement (MSP, paths, contexte org...), appelé par les scripts réseau.

Tableau 7: Scripts d'installation et d'orchestration (Traffic Core)

Script	Catégorie	Rôle principal	Commande typique
scripts/install-prerequisites.sh	Installation	Installer Docker/Go/Node...	./scripts/install-prerequisites.sh
scripts/install-fabric.sh	Installation	Installer Hyperledger Fabric	./scripts/install-fabric.sh
scripts/network.sh	Orchestration réseau	Démarrer réseau + déployer chaincodes	./scripts/network.sh up / deployCC
start-all.sh	Orchestration globale	Lancer tout le système	Lancer tout le système
stop-all.sh	Orchestration globale	Arrêter tout le système	./stop-all.sh
network/scripts/envVar.sh	Helper réseau	Variables d'environnement Fabric	

4.3. Déploiement des chaincodes en mode CCaaS

Les chaincodes (ou smart contracts) sont déployés sur la blockchain afin de gérer les différentes interactions entre les modules du système. Trois chaincodes ont été définis pour ce projet :

1. **road-manager** : Gestion des routes, de leur statut, et des événements de trafic.
2. **sensor-data** : Collecte des données de capteurs IoT et enregistrement des lectures.
3. **traffic-registry** : Gestion des informations relatives aux véhicules et aux conducteurs.

Les chaincodes sont déployés en mode **Chaincode-as-a-Service (CCaaS)**, ce qui permet une gestion facile des mises à jour et des versions de chaincode.

Tableau 8: Fonctions principales des chaincodes déployés

Chaincode	Fonction
road-manager	Créer une route, mettre à jour l'état des routes
sensor-data	Enregistrer les lectures des capteurs IoT
traffic-registry	Enregistrer un véhicule, gérer les violations

4.4. Implémentation détaillée des smart contracts

Dans Traffic Core, la logique métier est portée par **3 chaincodes** déployés en mode **CCaaS** : **road-manager**, **sensor-data** et **traffic-registry**. Ils permettent de gérer respectivement **les routes**, **les capteurs/lectures**, et **le registre véhicules/conducteurs + infractions**.

4.4.1. Chaincode road-manager (gestion des routes & trafic)

Objectif : gérer le cycle de vie des routes (création, consultation, mise à jour du statut) et stocker les informations de trafic associées (ex. données de densité/vitesse moyenne).

Traffic Core - Smart City Block...

Principales actions supportées :

- Initialiser des données de test (routes déjà existantes).
- Créer une route avec ses caractéristiques.
- Mettre à jour le statut d'une route (open/closed/maintenance/congested).
- Mettre à jour des données de trafic (ex. vitesse moyenne / flux).
- Interroger la blockchain (route par ID, toutes les routes, filtrage par statut).

Tableau 9: Fonctions du chaincode road-manager

Fonction	Rôle
InitLedger	Initialise des données de test
CreateRoad	Crée une nouvelle route
GetRoad	Récupère une route par ID
UpdateRoadStatus	Modifie le statut d'une route
UpdateTrafficData	Met à jour les données de trafic
GetAllRoads	Liste toutes les routes
GetRoadsByStatus	Filtre les routes par statut

4.4.2. Chaincode sensor-data (capteurs IoT & lectures temps réel)

Objectif : gérer l'enregistrement des capteurs IoT et la conservation des **lectures** (vitesse, météo, air quality, etc.) sous forme de transactions blockchain, et générer des **alertes**.

Principales actions supportées :

- Enregistrer un capteur (identité, type, localisation).

- Enregistrer une lecture en temps réel (lecture horodatée).
- Consulter l'historique d'un capteur.
- Créer des alertes et afficher les alertes actives

Tableau 10: Fonctions du chaincode sensor-data

Fonction	Rôle
InitLedger	Initialise des capteurs de test
RegisterSensor	Enregistre un nouveau capteur
RegisterSensor	Récupère un capteur par ID
GetSensor	Retourne l'historique des lectures
RecordReading	Crée une alerte
GetReadingsBySensor	Retourne les alertes actives
CreateAlert	Crée une alerte
GetActiveAlerts	Retourne les alertes actives

4.4.3. Chaincode traffic-registry (véhicules, conducteurs, infractions)

Objectif : gérer l'identification des véhicules/conducteurs et enregistrer les **violations** (ex. excès de vitesse).

Traffic Core - Smart City Block...

Principales actions supportées :

- Enregistrer un véhicule.
- Consulter un véhicule.
- Enregistrer un conducteur.
- Consulter un conducteur.
- Transférer la propriété du véhicule.
- Enregistrer une infraction.

Tableau 11: Fonctions du chaincode traffic-registry

Fonction	Rôle
InitLedger	Initialise des données de test
RegisterVehicle	Enregistre un véhicule
GetVehicle	Récupère un véhicule
RegisterDriver	Enregistre un conducteur
GetDriver	Récupère un conducteur
TransferVehicle	Transfert de propriété
RecordViolation	Enregistre une infraction

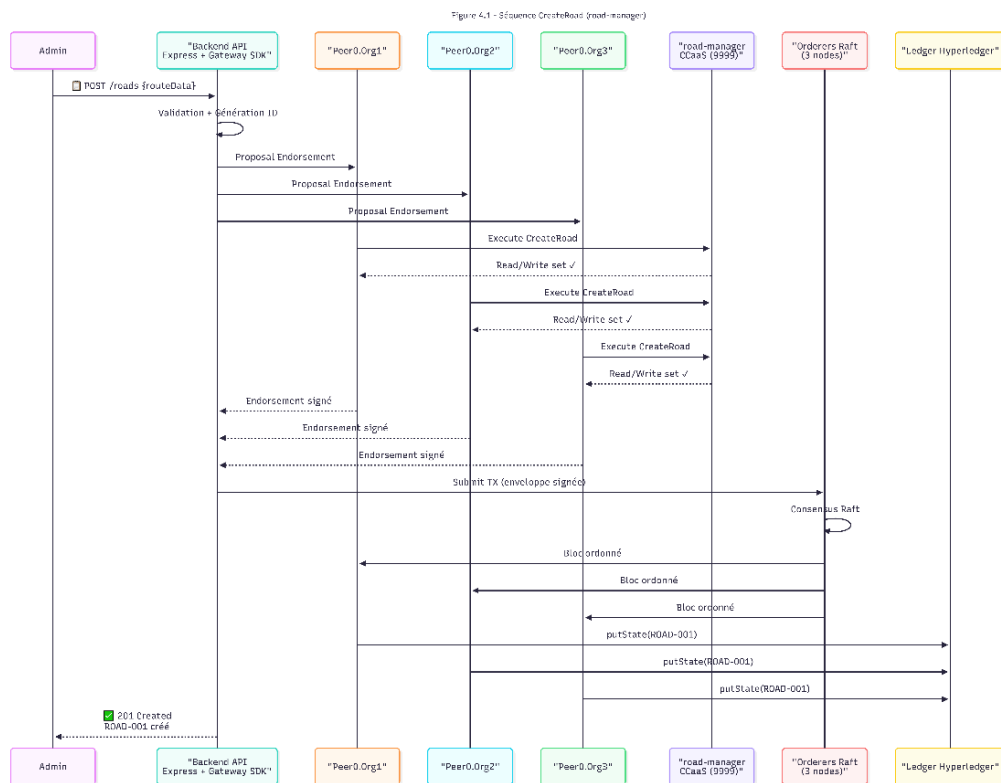


Figure 10: Diagramme de séquence d'une transaction CreateRoad

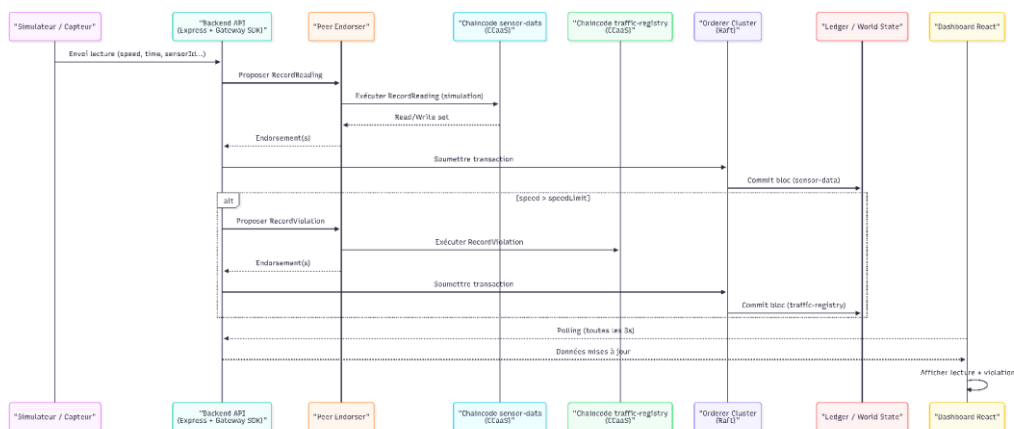


Figure 11: Diagramme de séquence d'une transaction RecordReading

4.5. Fonctionnement du simulateur et du tableau de bord

Cette partie décrit comment **les données de trafic sont générées, envoyées à la blockchain**, puis **visualisées en temps réel** via le dashboard. Dans votre projet, l'exécution se fait via **3 services applicatifs** (backend, frontend, simulateur) + le **réseau Hyperledger Fabric**.

4.5.1. Démarrage et accès à la plateforme

Deux modes sont possibles :

- **Démarrage complet (recommandé)** : lance tout automatiquement via un seul script.
- **Démarrage manuel** :
 - réseau Fabric + déploiement chaincodes
 - backend
 - frontend
 - simulateur

Accès au Dashboard

- URL : <http://localhost:5176>

4.5.2. Simulateur IoT (génération des données trafic)

Le **simulateur IoT** joue le rôle de “capteurs virtuels” : il génère des **lectures** (ex. vitesse) et les envoie au système pour qu'elles soient enregistrées via la blockchain. Il permet aussi de produire des situations réalistes grâce à des **scénarios**.

Fonctions principales du simulateur

- Génération automatique de **lectures capteurs** (traffic/speed/etc.).
- **Scénarios** de simulation : (ex. *rush hour*, *rainy*).
- Création de **violations automatiques** quand la **vitesse dépasse la limite** (speedLimit).
- Enregistrement et suivi via logs (le projet mentionne un log fichier côté simulateur).

4.5.3. Tableau de bord React (visualisation + contrôle)

Le dashboard est une interface React accessible via navigateur. Il regroupe plusieurs pages dédiées à chaque sous-module, et affiche des données **en temps réel** (notamment les violations).

Pages principales disponibles

- **Dashboard** : vue globale + “Live Violations Feed” (rafraîchissement toutes les 3 secondes).
- **Roads** : affichage des routes et intersections (routes/intersections prédéfinies dans le projet).
- **Sensors** : liste et informations des capteurs IoT.
- **Registry** : véhicules et conducteurs.
- **Simulation** : contrôle simulation + logs blockchain.
- **Consensus** : tests PBFT vs PoA (page dédiée aux expérimentations).
- **Security** : SOC (4 agents IA, activation et simulation d’attaques)

4.5.4. Mise à jour temps réel et intégration blockchain

Le fonctionnement “temps réel” côté utilisateur se fait ainsi :

1. **Le simulateur** génère une lecture (ex. vitesse).
2. **Le backend** soumet la transaction à Fabric (chaincode capteurs / registre).
3. Dès qu’une violation est détectée (vitesse > limite), elle est enregistrée.
4. **Le dashboard** affiche automatiquement la nouvelle violation via rafraîchissement régulier (toutes les 3 secondes)

5. Architecture de Sécurité et SOC Intelligent

Ce chapitre présente la **sécurisation** de la plateforme Traffic Core et l'intégration d'un **SOC (Security Operations Center)** intelligent. L'objectif est d'assurer **l'authentification**, la **confidentialité**, **l'intégrité** et la **traçabilité** des actions (trafic, capteurs, registre), tout en **détectant** et **répondant** aux attaques simulées.

5.1. Exigences de sécurité pour Traffic Core

Dans **Traffic Core**, les données manipulées (routes/intersections, lectures capteurs, registre véhicules/conducteurs, infractions) sont **critiques** : elles doivent rester **fiables**, **protégées** et **auditable**s. Pour cela, tu as mis en place une sécurité à deux niveaux :

1. **Sécurité “native Fabric”** (réseau blockchain + contrôle d’accès)
2. **Sécurité opérationnelle SOC** (détection, analyse, réponse, traçabilité)

5.1.1. Confidentialité et sécurisation des communications

L’exigence principale est d’éviter l’interception ou la modification des échanges entre composants (peers, orderers, clients).

- **TLS activé** sur les communications des peers et orderers (transport chiffré).

Traffic Core - Smart City Block...

- Cette protection est cohérente avec ton réseau multi-nœuds (plusieurs peers + orderers), où le trafic réseau est permanent.

5.1.2. Authentification et gestion des identités

Dans une Smart City, il faut garantir que seuls des acteurs légitimes (organisations, services, capteurs) peuvent agir.

- Utilisation de **certificats X.509** pour l’identité des entités.
- Gestion par organisation via **MSP (Membership Service Provider)**.
- Ton réseau est organisé en **3 organisations** (TrafficAuthority / MobilityServices / SensorNetwork), ce qui permet de séparer responsabilités et identités.

5.1.3. Autorisation et contrôle des transactions (policies)

Ici, l’objectif est d’empêcher qu’une organisation non autorisée valide ou manipule certaines opérations.

- Utilisation de **Policies configurables** (au niveau chaincode et réseau).

- Politique d'endorsement mise en place en **OR** : une seule organisation peut endosser/valider la transaction (choix utile pour fluidité dans une simulation).

5.1.4. Intégrité et traçabilité des données

L'exigence ici est : "toute action doit être traçable et non falsifiable".

- Les opérations métier sont enregistrées sous forme de **transactions** via les 3 chaincodes (routes, capteurs, registre).
- La plateforme enregistre aussi les **violations** (excès de vitesse) avec calcul automatique d'amende et points, ce qui renforce l'auditabilité.

5.1.5. Disponibilité et résilience (attaque, surcharge, pannes)

Dans une Smart City, un système indisponible = système inutilisable. Tu as donc prévu la capacité à détecter et réagir à des attaques qui visent la disponibilité :

- Support de la **simulation d'attaques** (ex. DDoS) pour tester la robustesse.
- Checklist de vérification : "SOC activable/désactivable" + "simulation d'attaque fonctionne".

5.1.6. Surveillance et réponse aux incidents (SOC)

En plus de Fabric, tu as ajouté un **SOC** pour surveiller, analyser et répondre :

- SOC composé de **4 agents** : Defender, Sensor, Analyzer, Controller.
- Le SOC peut déclencher une réponse active, par exemple **blocage IP**.
- Les **incidents sont enregistrés sur la blockchain** (traçabilité sécurité).

5.1.7. Analyse intelligente (LLM) pour qualification des alertes

Tu as intégré une analyse avancée pour mieux classer et prioriser les menaces :

- **Mistral 7B Instruct** via **LM Studio** (serveur local) utilisé par l'Analyzer pour enrichir l'analyse.
- Possibilité d'analyse d'événements (ex. `transaction_flood`) avec score et actions recommandées

5.1.8. Pilotage et vérification (API + dashboard)

Enfin, l'exigence est de pouvoir **activer**, **contrôler** et **vérifier** la sécurité facilement :

- Endpoints backend pour le SOC (status/start).
- Endpoints SOC complets (agents, simulate attack, config LLM, analyze).

5.2. Architecture du Security Operations Center (SOC)

Dans un **Traffic Core**, le **SOC (Security Operations Center)** est un module de sécurité ajouté pour compléter la sécurité “native” d'Hyperledger Fabric. Son rôle est de **surveiller**, **détecter**, **analyser** et **répondre** aux incidents en temps réel, tout en assurant une **traçabilité** des événements (enregistrement des incidents sur la blockchain).

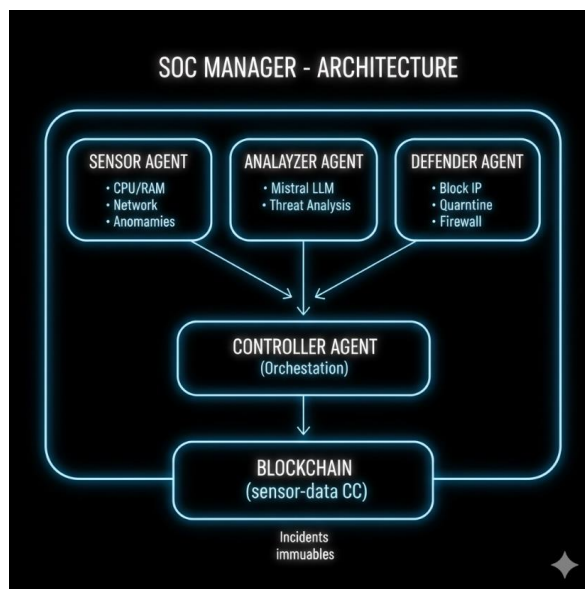


Figure 12: Architecture de SOC

5.2.1. Objectifs du SOC dans Traffic Core

Le SOC a été conçu pour répondre à trois besoins essentiels :

- **Surveillance continue** de la plateforme (trafic réseau, comportements anormaux, pics de transactions).
- **Analyse intelligente** des alertes et incidents (détection + scoring + recommandations).
- **Réponse automatisée** : déclencher des actions correctives comme le **blocage d'IP** ou la génération d'alertes et d'incidents.

5.2.2. Composants de l'architecture SOC (4 agents)

SOC est structuré autour de **4 agents**, chacun ayant une responsabilité claire :

1. Sensor Agent (Monitoring)

- Rôle : **collecte et monitoring**.
- Il observe des signaux comme : fréquence anormale de requêtes, saturation, comportement suspect.

2. Analyzer Agent (Analyse + IA)

- Rôle : **analyse approfondie** des événements détectés.
- Il peut intégrer une analyse intelligente via un **LLM** (voir 5.2.3).

3. Defender Agent (Réponse active)

- Rôle : exécuter une **réponse automatique**.
- Exemple d'action implémentée : **blocage IP** en cas d'attaque ou activité suspecte.

4. Controller Agent (Orchestration)

- Rôle : coordination générale : il relie les signaux détectés, l'analyse, et la réponse.
- Il déclenche le bon workflow (alerte, incident, action defender).

5.2.3. Intégration de l'IA (LLM) dans l'Analyzer

Pour enrichir la détection "classique" (seuils + règles), vous avez intégré un modèle LLM :

- Modèle : **Mistral 7B Instruct**
- Hébergement : via **LM Studio** (serveur local OpenAI-compatible).
- Paramètres utilisés : temperature, max_tokens, top_p (configurables).

Ce que l'Analyzer produit (exemple réel) :

- **risk_score**
- **analysis** (diagnostic)

- **recommended_actions** (actions recommandées)

5.2.4. Interfaces de contrôle (API + Dashboard)

Le SOC est piloté depuis deux interfaces :

A) API backend

- Contrôle minimal (démarrage et état) :
 - GET /api/v1/soc/status
 - POST /api/v1/soc/start
- API SOC complète (pilotage + simulation + IA) :
 - GET /soc/status
 - POST /soc/start
 - GET /soc/agents
 - POST /soc/simulate-attack
 - POST /soc/analyze
 - POST /soc/config/llm

B) Dashboard

- Page **Security** dédiée :
 - activation SOC
 - simulation d'attaques
 - visualisation des agents et alertes

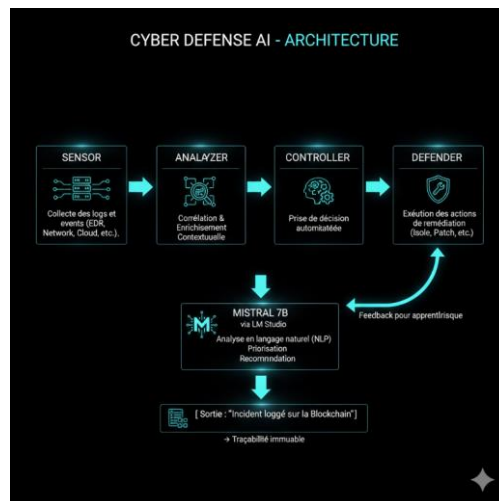


Figure 13: Architecture du SOC (Defender / Controller / Sensor / Analyzer)

5.3. Intégration du SOC avec Hyperledger Fabric

L'intégration du **SOC** dans votre projet Traffic Core vise à connecter la **surveillance sécurité** (détection + analyse + réponse) avec le cœur de la plateforme **Hyperledger Fabric**, afin d'obtenir deux résultats essentiels :

1. **Réagir** rapidement aux comportements suspects (attaques simulées, anomalies).
2. **Tracer** les incidents de sécurité de manière **immuable** (audit sur blockchain).

5.3.1. Points d'intégration (où le SOC se branche)

Dans un architecture, le SOC est intégré à la plateforme à travers :

- **Le Backend API (Express.js)** : c'est le point central qui reçoit les alertes, lance les analyses et déclenche des actions.
 - Endpoints disponibles :
 - GET /api/v1/soc/status
 - POST /api/v1/soc/start
 - Et une API SOC plus complète :
 - /soc/status, /soc/start, /soc/agents, /soc/simulate-attack, /soc/analyze, /soc/config/llm
- **Le Dashboard React** : interface de pilotage et visualisation, via la page **Security** (activation SOC, simulation attaques, suivi agents).

- **La Blockchain Fabric** : utilisée comme “registre d’audit” pour journaliser les incidents et garantir qu’ils ne peuvent pas être modifiés.

5.3.2. Journalisation (audit) des incidents sur la blockchain

Un des choix importants de votre projet est de **stocker les incidents SOC sur la blockchain**, afin de :

- garantir l’**intégrité** des preuves (logs immuables),
- permettre un audit complet : *qui a déclenché l’alerte ? quand ? quelles actions ont été prises ?*
- assurer une traçabilité partagée entre organisations (vision commune des incidents).

5.3.3. Détection d’événements liés à Fabric / réseau (exemple concret)

Le SOC surveille des événements qui impactent directement la plateforme. Exemple typique :

- **Transaction flood (transaction_flood)** : hausse brutale du volume de transactions (symptôme DDoS ou comportement anormal).

Une fois l’événement capté par le **Sensor Agent**, il est transmis à l’Analyzeur qui :

- attribue un **risk_score**,
- fournit une **analysis**,
- propose des **recommended_actions** (ex. throttling, blocage IP...).

5.3.4. Interaction SOC ↔ réponse active (actions défensives)

Après analyse, le pipeline SOC permet une réponse automatique :

- **Defender Agent** peut exécuter des actions comme le **blocage IP** (réponse active).
- Cette capacité est citée comme mécanisme de protection SOC dans votre projet.

Cette réponse est ensuite **journalisée** pour garantir l’auditabilité de la décision.

5.3.5. Intégration IA (LLM) pour enrichir la sécurité

SOC est enrichi par un modèle IA :

- **Mistral 7B Instruct** hébergé via **LM Studio** (serveur local).
- L'Analyzer l'utilise pour :
 - produire une explication plus claire,
 - prioriser les incidents,
 - recommander des actions (mitigation).

5.4. Modèle de menaces et attaques simulées

Dans le contexte de Traffic Core, le Security Operations Center (SOC) permet de surveiller les événements de sécurité et de répondre aux attaques. Afin de tester la résilience du système, plusieurs attaques simulées sont mises en œuvre pour simuler des menaces réelles auxquelles la plateforme pourrait être confrontée. Ces attaques sont basées sur des scénarios courants qui affectent les systèmes de blockchain permissionnée et les réseaux urbains intelligents.

Les types de menaces et attaques simulées comprennent des attaques Sybil, 51%, Replay, Eclipse, et DDoS. Ces attaques sont détectées et analysées par le SOC à l'aide des 4 agents : Sensor, Analyzer, Defender, et Controller. Chaque type d'attaque dispose d'indicateurs spécifiques permettant à l'Analyzer (avec le modèle LLM de Mistral 7B Instruct) de qualifier l'incident, d'attribuer un score de risque, et de recommander des actions. Les attaques simulées et leurs caractéristiques

Les attaques simulées sont choisies en fonction des scénarios courants dans les environnements de blockchain permissionnée et les réseaux de Smart Cities. Voici une description de chaque type d'attaque simulée et les indicateurs observables qui aident le SOC à les détecter.

1. Attaque Sybil

- **Objectif** : Créer de fausses identités pour prendre le contrôle de la majorité des nœuds du réseau.
- **Indicateurs observables** :
 - **Enregistrements rapides** de nœuds fictifs.

- **Même source d'adresse IP.**
- **Comportement répétitif** dans les transactions.

2. Attaque 51%

- **Objectif** : Contrôler plus de 50% de la puissance de calcul du réseau pour réorganiser la chaîne.
- **Indicateurs observables** :
 - **Réorganisation de la chaîne** (différence entre le bloc actuel et les transactions validées).
 - **Anomalies de blocs** : bloc valide rejeté par la majorité, mais accepté par 51% des nœuds.

3. Attaque Replay

- **Objectif** : Rejouer une transaction légitime mais frauduleuse.
- **Indicateurs observables** :
 - **Signatures dupliquées** dans les transactions.
 - **Timestamps invalides** (transactions en dehors de la fenêtre de validité).

4. Attaque Eclipse

- **Objectif** : Isoler certains nœuds en manipulant leur visibilité du réseau.
- **Indicateurs observables** :
 - **Connexions monopolisées** avec un même nœud, réduisant la visibilité du réseau.
 - **Comportement réseau anormal** : perte de la synchronisation des blocs.

5. Attaque DDoS (Distributed Denial of Service)

- **Objectif** : Surcharger le réseau pour provoquer une dégradation du service ou un arrêt complet.

- **Indicateurs observables :**

- **Tx/sec élevés** et une augmentation soudaine de la charge.
- **Épuisement des ressources** (consommation CPU ou bande passante excessive).

5.4.1. Détection et réponse aux attaques

Chaque type d'attaque est détecté par le **Sensor Agent**, qui **surveille les signaux d'anomalie** dans les transactions et les comportements du réseau. Une fois l'attaque détectée, elle est transmise à l'**Analyzer Agent** pour analyse approfondie et qualification de l'incident.

Le **Controller Agent** orchestre la réponse, en décidant des actions appropriées selon le type et la gravité de l'attaque. Par exemple, si une attaque **DDoS** est détectée, le Defender Agent peut décider de **bloquer certaines IP** ou de **limiter le nombre de connexions** pour éviter une surcharge. Si une attaque **Sybil** est détectée, le Defender peut initier une vérification des nouveaux nœuds du réseau pour identifier des fausses identités. Chaque étape du processus est enregistrée sur la blockchain pour assurer la **traçabilité** des actions prises.

5.4.2. Cartographie des attaques vers les composants SOC

Les attaques simulées sont cartographiées dans un diagramme qui montre comment chaque type d'attaque peut affecter les différents composants du **SOC**. Ce diagramme illustre les **flux d'information** entre les agents de sécurité, la plateforme blockchain et les résultats observés après l'analyse des attaques. Il permet de visualiser les **menaces** et de comprendre comment elles sont traitées par les différents agents.

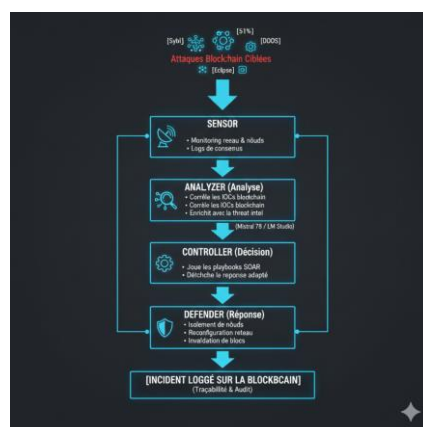


Figure 14: Cartographie des attaques vers composants SOC

6. Expérimentation, Consensus et Résultats

Ce chapitre présente les expérimentations menées sur le système **Traffic Core**, en se concentrant sur les **mécanismes de consensus** testés : **Proof of Authority (PoA)** et **Practical Byzantine Fault Tolerance (PBFT)**. Nous détaillons la méthodologie expérimentale, comparons les performances des deux algorithmes, et présentons les résultats obtenus sous forme de mesures de latence, débit, résilience et consommation de ressources. Enfin, nous illustrons les fonctionnalités du système à travers des captures d'écran du tableau de bord et des interfaces opérationnelles.

6.1 Méthodologie expérimentale

Les tests de consensus ont été réalisés dans un environnement contrôlé avec le réseau Hyperledger Fabric configuré avec 3 organisations, 6 peers et 3 orderers. Deux configurations distinctes ont été déployées :

- **Configuration PoA** : Un seul nœud ordonnateur désigné comme autorité valide les blocs.
- **Configuration PBFT** : Plusieurs nœuds ordonnateurs participent à un vote pour valider chaque bloc (tolérance byzantine).

Métriques mesurées :

- **Latence** : Temps entre l'émission et la validation d'une transaction.
- **Débit** : Nombre de transactions par seconde (TPS) traitées.
- **Résilience** : Capacité à fonctionner en présence de nœuds défaillants ou malveillants.
- **Consommation des ressources** : Utilisation CPU, mémoire et réseau.

Scénarios de test :

- Envoi de transactions simples (création de routes, enregistrement de capteurs).
- Simulation de pics de charge (rush hour simulé).
- Injection de nœuds défaillants (pour PBFT).

6.2 Comparaison PoA vs PBFT

6.2.1 Latence moyenne

Type de transaction	PoA (ms)	PBFT (ms)
Création de route	120	320
Lecture capteur	95	280
Enregistrement violation	150	400

Observation : PoA est significativement plus rapide grâce à sa simplicité (validation par une seule autorité). PBFT, plus sécurisé, introduit un surcoût dû aux phases de vote et de synchronisation.

6.2.2 Débit maximal (TPS)

Mécanisme	TPS moyen (pic)
PoA	450
PBFT	180

Observation : PoA permet un débit plus élevé, adapté aux scénarios nécessitant une haute fréquence de transactions (ex. lecture de capteurs en temps réel). PBFT, bien que plus lent, garantit une validation distribuée et résistante aux fautes byzantines.

6.2.3 Résilience aux fautes

- **PoA :** Vulnérable si l'autorité unique est compromise.
- **PBFT :** Tolère jusqu'à $\lfloor (N-1)/3 \rfloor$ nœuds défaillants (dans notre cas, 1 nœud sur 3).

Test d'injection de faute :

- Avec PBFT, le réseau a continué à fonctionner normalement après la défaillance d'un orderer.
- Avec PoA, la panne de l'unique orderer a bloqué toute validation.

6.2.4 Consommation des ressources

Mécanisme	CPU moyen (%)	Mémoire (MB/peer)
PoA	15–25	180–220
PBFT	35–50	250–300

Observation : PBFT est plus gourmand en ressources en raison de la communication accrue entre les nœuds pour parvenir au consensus.

6.3 Résultats et analyse

6.3.1 Choix du consensus selon le cas d'usage

- **PoA** est recommandé pour :
 - Les environnements de confiance (organisations connues).
 - Les applications nécessitant une faible latence et un haut débit.
 - Les simulations en temps réel où la rapidité prime.
- **PBFT** est préférable pour :
 - Les environnements à risque de compromission ou de défaillance.
 - Les applications critiques où l'intégrité et la sécurité sont prioritaires.
 - Les scénarios réglementés exigeant une validation distribuée.

6.3.2 Impact sur Traffic Core

Dans **Traffic Core**, nous avons opté pour une **configuration hybride** :

- **Raft** (dérivé de PBFT) pour la gestion des ordres de transaction (orderers).
- **Politique d'endorsement simplifiée (OR)** pour valider les transactions rapidement tout en maintenant une décentralisation modérée.

Ce choix permet de concilier performance et sécurité, adapté aux besoins d'une Smart City où les données de trafic doivent être à la fois rapides et fiables.

6.4 Captures d'écran de l'application

Voici quelques illustrations des interfaces opérationnelles de **Traffic Core** :

6.4.1 Tableau de bord principal

Vue globale du système avec indicateurs clés et visualisations



Figure 15: Tableau de bord - Métriques et statistiques

Affichage des KPI principaux : routes, véhicules, conducteurs, capteurs, violations et alertes SOC.

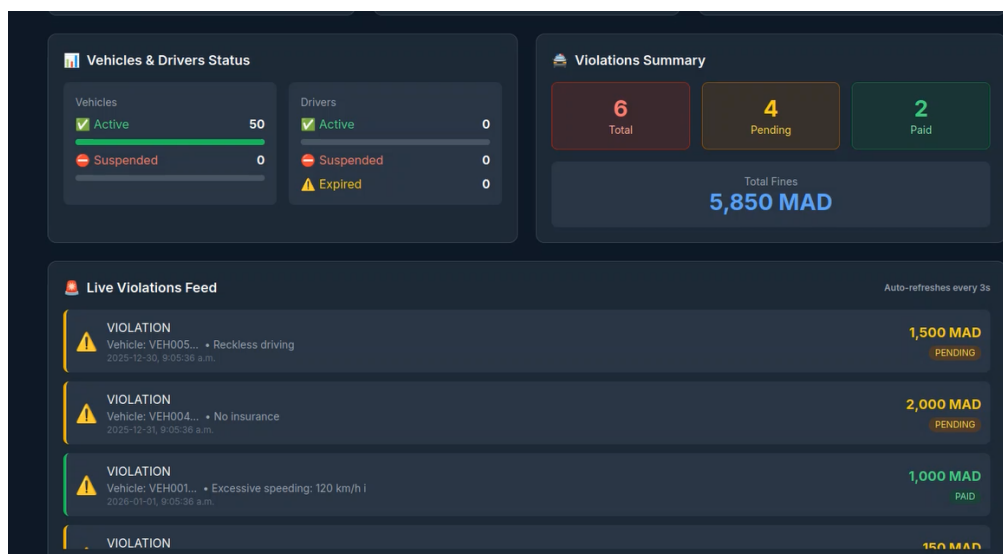


Figure 16: Tableau de bord - Visualisations détaillées

Sections complémentaires montrant : Statut des routes, types de véhicules, types de carburant, statut véhicules/conducteurs, résumé des violations et flux temps réel des infractions.

6.4.2 Page de simulation du trafic

Interface complète de contrôle, visualisation et monitoring du trafic urbain simulé

Panneau de démarrage de la simulation avec indicateurs clés



Figure 17: Simulation - Panneau de contrôle principal

Visualisation cartographique du réseau routier montrant les intersections (Int-1 à Int-8), les routes principales (Avenue Mohammed V, Boulevard Hassan II) et les véhicules en mouvement. Légende des types de véhicules : Voiture, Camion, Bus, Moto. Données temps réel : densité (véh/h), vitesse (km/h), parking (places), température (°C).



Figure 18: Simulation - Carte interactive du trafic urbain

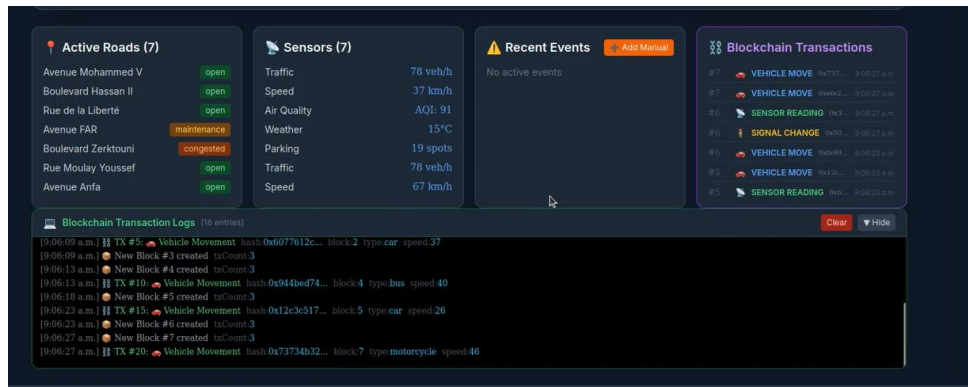


Figure 19: Simulation - Tableau de bord détaillé

Interface divisée en 4 sections :

1. **Routes actives** - Liste des 7 artères principales
2. **Capteurs IoT** - Mesures en temps réel (trafic: véh/h, vitesse: km/h, météo: °C)
3. **Journaux blockchain** - Transactions et création de blocs avec horodatage
4. **Transactions récentes** - Visualisation graphique des opérations (mouvements véhicules, lectures capteurs)

6.4.3 Page de gestion du réseau routier

Interface de configuration et monitoring des infrastructures routières urbaines sur blockchain

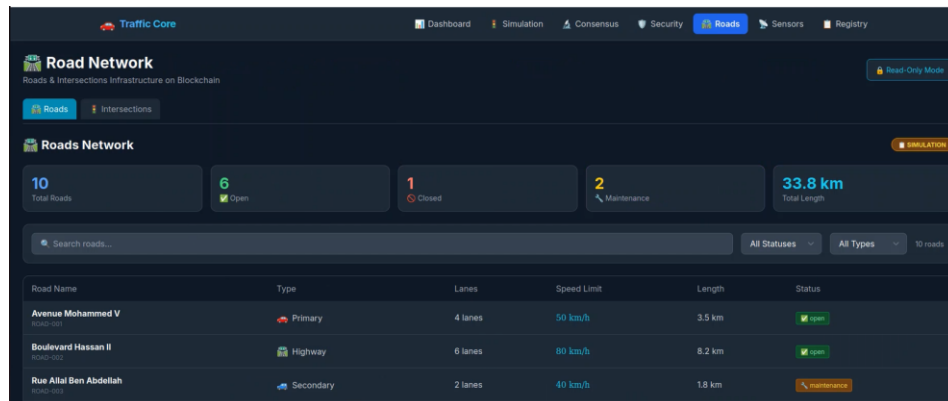


Figure 20: Routes - Vue d'ensemble du réseau routier

Tableau de bord principal du module routier montrant l'infrastructure complète gérée sur la blockchain Hyperledger Fabric. Navigation par onglets "Roads" et "Intersections" avec fonctionnalités de recherche et filtrage avancées pour une gestion optimisée du trafic urbain.

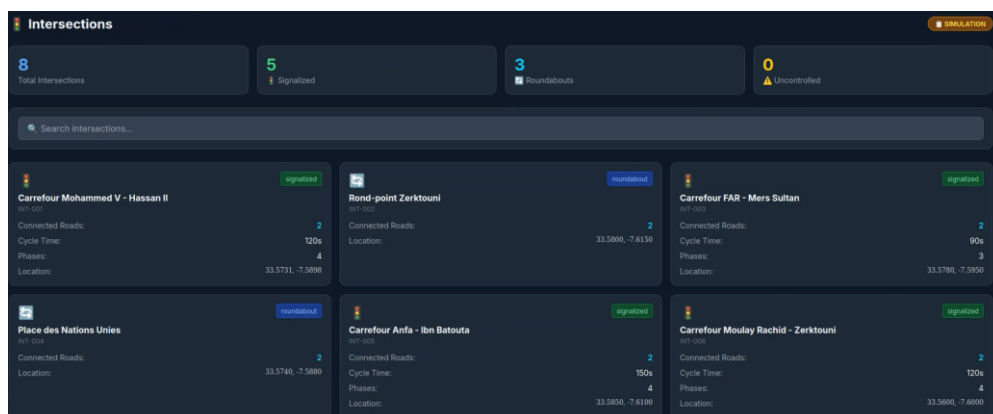


Figure 21: Routes - Détail des intersections intelligentes

Configuration détaillée des intersections urbaines avec paramètres techniques avancés :

- **Carrefour Mohammed V - Hassan II** : 2 routes connectées, cycle de feux de 60 secondes, 4 phases de régulation
- **Rond-point Zerkkouni** : 3 routes connectées, emplacement géolocalisé précis
- **Carrefour FAR - Mers Sultan** : 2 routes connectées, cycle de 90 secondes, 3 phases de circulation

6.4.4 Monitoring des capteurs IoT

Surveillance en temps réel du réseau de capteurs urbains déployés dans la Smart City

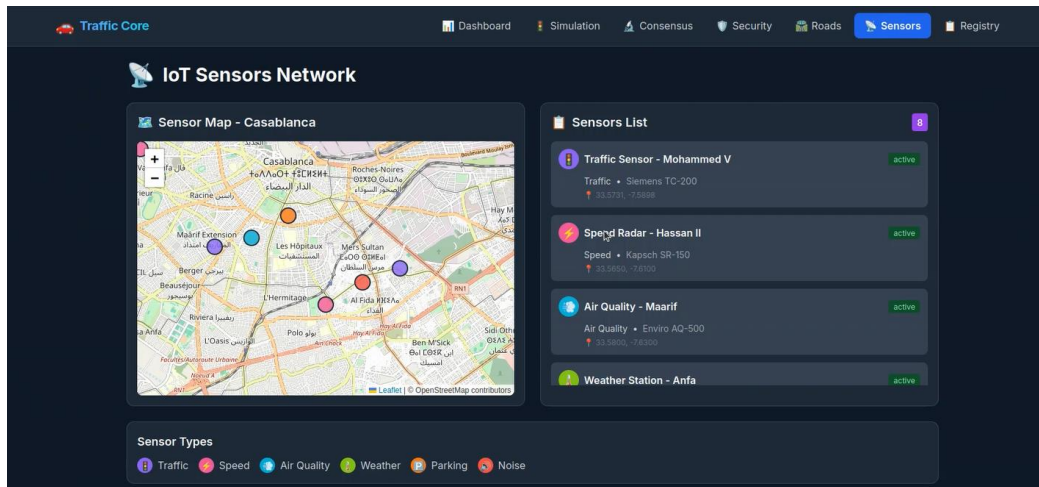


Figure 22: Capteurs - Réseau IoT de CASABLANCA

Interface complète de monitoring des capteurs intelligents comprenant :

1. **Carte géographique interactive** - Visualisation spatiale des capteurs déployés dans différents quartiers de Casablanca
2. **Liste détaillée des capteurs** - Fiche technique pour chaque dispositif :
 - Capteur Trafic Mohammed V (Siemens TC-200) : position 33.3731, 7.9388
 - Radar de vitesse Hassan II (Kapsch SR-150) : position 33.3560, 7.6900
 - Qualité d'air Maarif (Enviro AG-500) : position 33.3600, 7.6900
 - Station météo Anfa : statut actif
3. **Classification par types** - Organisation des capteurs selon leur fonction : trafic, vitesse, qualité d'air, météo, parking et bruit. Données collectées en temps réel et enregistrées sur la blockchain pour une traçabilité complète.

6.4.5 Registre du trafic sur blockchain

Gestion centralisée des véhicules, conducteurs et violations avec enregistrement immuable

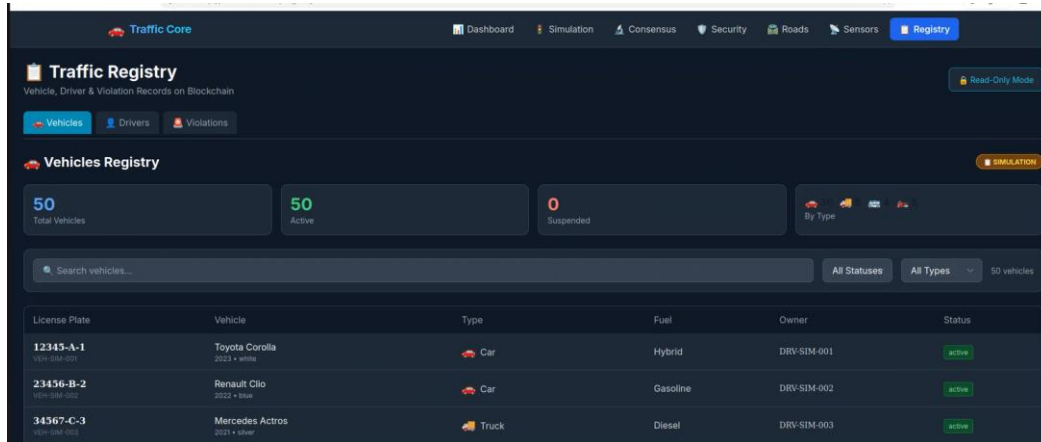


Figure 23: Registry - Gestion des véhicules

Registre blockchain des véhicules urbains avec 50 véhicules actifs. Interface comprenant :

- Tableau des véhicules avec plaque d'immatriculation (ex: 12343-A-1), marque (Toyota, Renault, Mercedes), type (voiture, camion), carburant (hybride, essence, diesel)
- Filtres avancés par statut, type et recherche textuelle
- Visualisation des statistiques : 50 véhicules totaux, tous actifs, classification par type

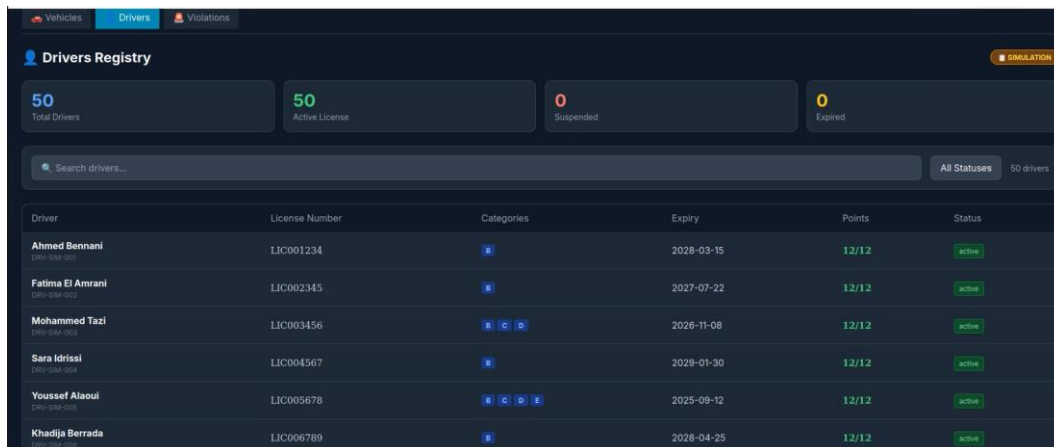


Figure 24: Registry - Gestion des conducteurs

Base de données des conducteurs enregistrés sur la blockchain avec informations d'identification uniques :

- Identifiants conducteurs (DRV-SIN-001, DRV-SIN-002, DRV-SIN-003)
- Gestion des permis de conduire et statuts d'activité

- Intégration avec le registre des véhicules pour lier propriétaires et conducteurs*

ID	Type	Vehicle	Driver	Fine	Points	Status
VI0001	Speeding	VEH001	DRV001	500 MAD	-	pending
VI0002	Red Light	VEH002	DRV002	700 MAD	-	paid
VI0003	Parking	VEH003	DRV003	150 MAD	-	pending
VI0004	Speeding	VEH001	DRV001	1000 MAD	-	paid
VI0005	No Insurance	VEH004	DRV004	2000 MAD	-	pending
VI0006	Reckless Driving	VEH005	DRV005	1500 MAD	-	pending

Figure 25: Registry - Détection et suivi des violations

Module de gestion des infractions au code de la route avec :

- Liste des violations détectées automatiquement (excès de vitesse, feux rouges, etc.)
- Calcul automatique des amendes selon le barème marocain (400-3000 MAD)
- Déduction des points de permis et suivi du statut des infractions (en attente, payée, contestée)
- Traçabilité complète sur la blockchain avec horodatage et preuves immuables

6.4.6 Security Operations Center (SOC) intelligent

Centre d'opérations de sécurité avancé avec détection IA et réponse automatisée

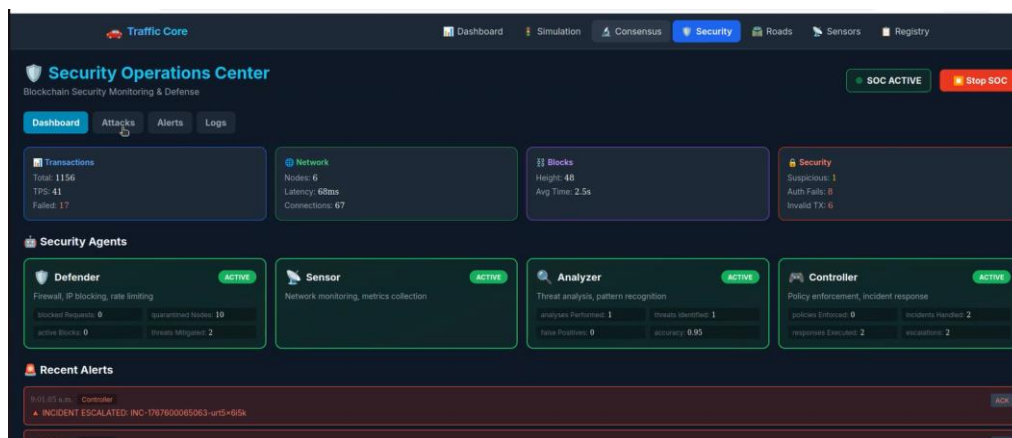


Figure 26: Security - Tableau de bord SOC

Vue d'ensemble du Security Operations Center opérationnel (SOC ACTIVE) avec :

- Métriques transactionnelles : 1156 transactions totales, 41 TPS, 17 échecs

- Statut des 4 agents IA : Defender (pare-feu, blocage IP), Sensor (monitoring réseau), Analyzer (analyse IA), Controller (orchestration)
- Réseau blockchain : 6 nœuds actifs, 67 connexions, surveillance en temps réel
- Alertes récentes : Détection d'incidents avec codes d'identification uniques (INC-1707000050063)
- Indicateurs de sécurité : 1 menace en escalade, 8 attaques simulées, 6 transactions suspectes
- Performance IA : Analyseur opérationnel avec reconnaissance de patterns avancée

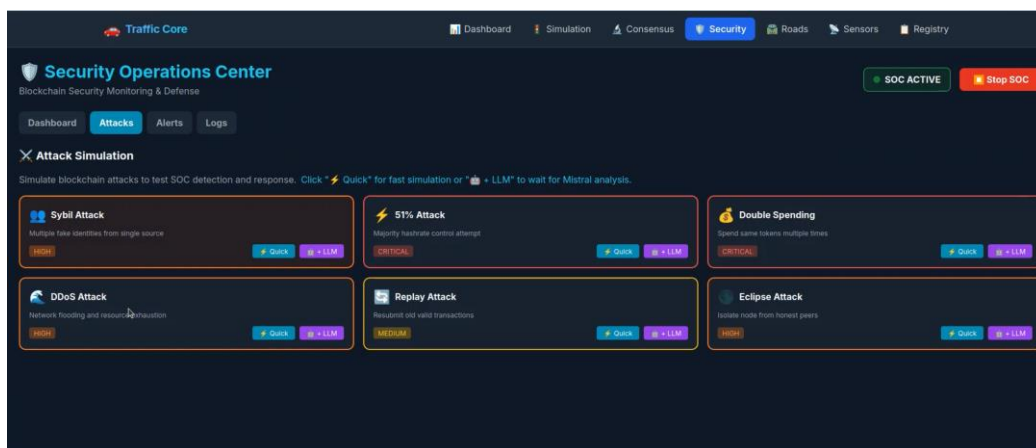


Figure 27: Security - Simulation d'attaques

Interface de simulation des menaces cybernétiques avec 6 types d'attaques configurables :

- **Attaque Sybil** : Identités multiples depuis une source unique
 - **Attaque DDoS** : Inondation réseau et épuisement des ressources
 - **Attaque 51%** : Tentative de contrôle majoritaire
 - **Double dépense** : Réutilisation frauduleuse de jetons
 - **Attaque Replay** : Réutilisation de transactions valides
 - **Attaque Eclipse** : Isolation des nœuds honnêtes
- Options de simulation rapide ou analyse approfondie par IA

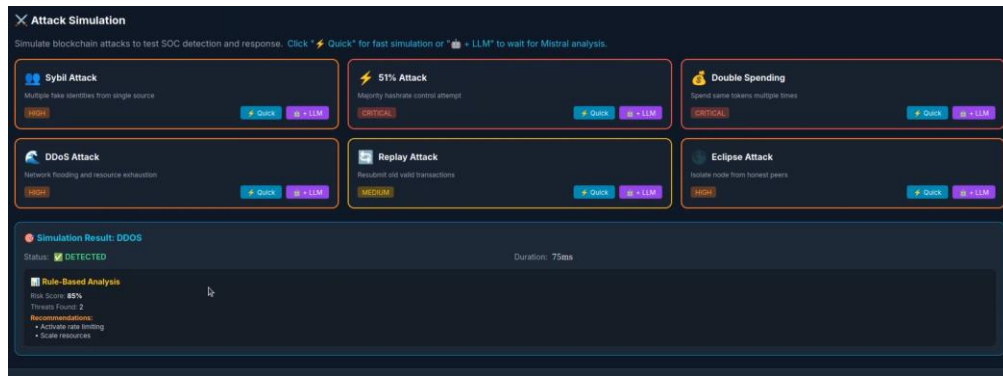


Figure 28: Security - Résultats d'analyse d'attaque

Résultats détaillés de simulation avec détection automatique et recommandations IA :

- **Attaque DDoS détectée** avec code incident #86
- **Analyse par rôles** et recommandations spécifiques
- **Actions proposées** : Atténuation du trafic, mise à l'échelle des ressources
- **Intégration Mistral LLM** pour l'analyse contextuelle avancée

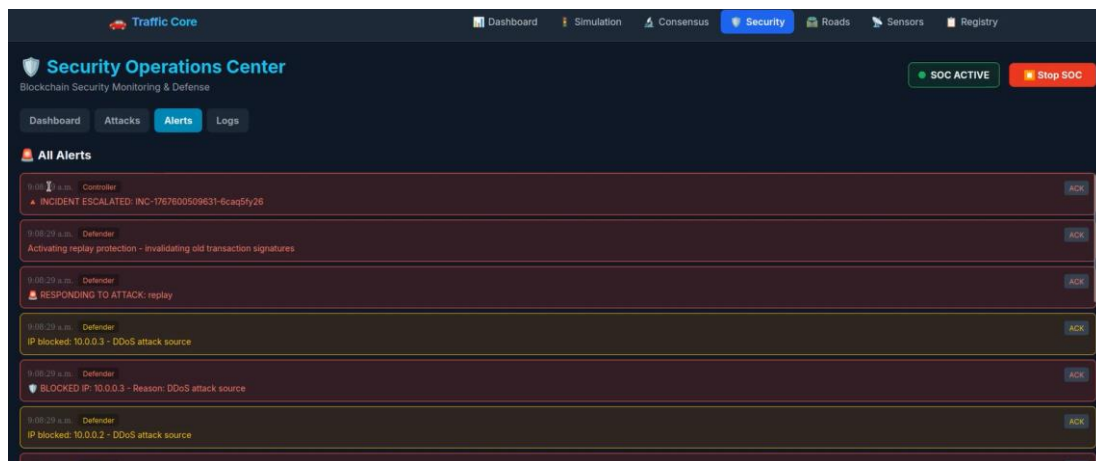


Figure 29: Security - Gestion des alertes

Tableau de bord des alertes de sécurité avec :

- Classification par sévérité (critique, haute, moyenne, basse)
- Alertes actives et historiques
- Assignment aux agents SOC
- Workflow de traitement et résolution

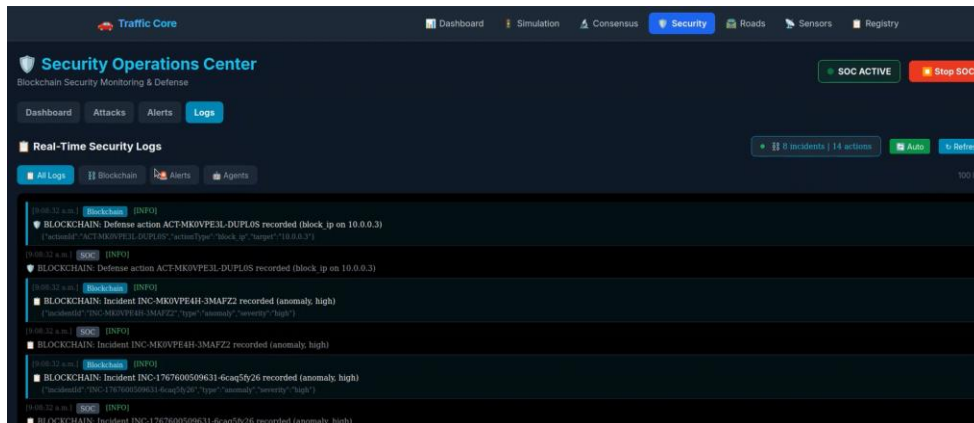


Figure 30: Security - Journaux de sécurité temps réel

Terminal de logs de sécurité avec :

- Flux en temps réel des événements de sécurité
- Corrélation entre alertes et actions défensives
- Traçabilité complète sur blockchain des incidents
- Intégration avec les agents Defender pour le blocage IP automatique*

6.4.7 Tests et comparaison des mécanismes de consensus

Analyse expérimentale des algorithmes PBFT et PoA dans un environnement blockchain permissionné

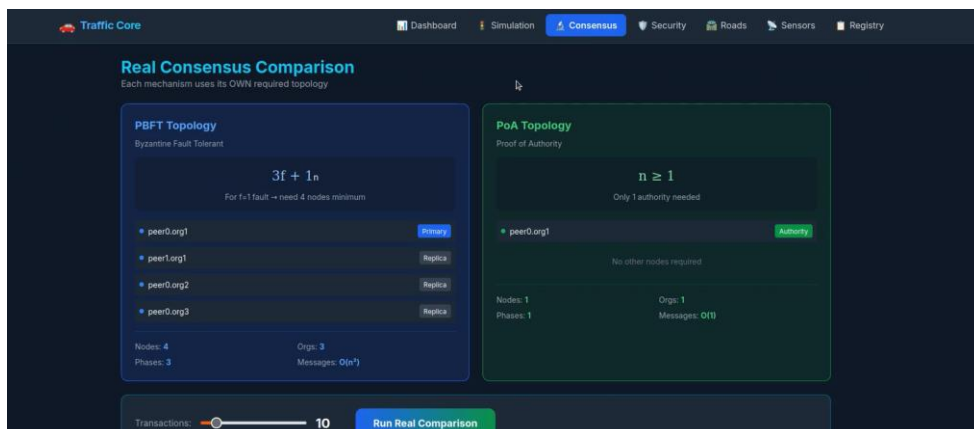


Figure 31: Consensus - Configuration des topologies

Interface de configuration comparative des deux mécanismes de consensus :

- **Topologie PBFT** : Modèle Byzantine Fault Tolerant nécessitant 4 nœuds minimum (formule $3f+1$), impliquant 3 organisations avec messagerie quadratique $O(n^2)$
- **Topologie PoA** : Modèle Proof of Authority avec seulement 1 nœud autoritaire requis, architecture centralisée avec messagerie constante $O(1)$
Paramètres configurables pour l'exécution de tests comparatifs avec 10 transactions

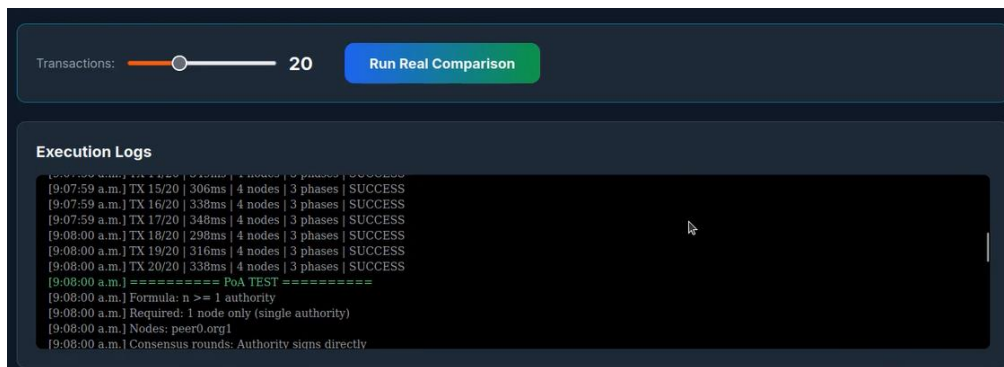


Figure 32: Consensus - Résultats comparatifs détaillés

Tableau synthétique des performances mesurées :

- **Débit** : PoA (20.86 TPS) 7 fois plus rapide que PBFT (2.98 TPS)
- **Latence** : PoA (47 ms) 86% plus rapide que PBFT (335 ms)
- **Tolérance aux fautes** : PBFT supporte 1 nœud byzantin vs PoA (aucune tolérance)
- **Décentralisation** : PBFT (haute) vs PoA (faible)
- **Taux de réussite** : 100% pour les deux mécanismes

Metric	PBFT (4 nodes)	PoA (1 node)	Difference
Required Nodes	4 ($3f+1$)	1 (authority)	3 more
Consensus Rounds	3	1	2 more
Throughput	2.98 TPS	20.86 TPS	PoA 7.0x faster
Avg Latency	335 ms	47 ms	PoA 86% faster
Success Rate	100.0%	100.0%	Equal
Fault Tolerance	$f < n/3$ (1 Byzantine node)	None (single authority)	PBFT more secure
Decentralization	High (multi-org)	Low (centralized)	PBFT more decentralized

PBFT - Best for:

- ✓ High security requirements
- ✓ Byzantine fault tolerance needed
- ✓ Multi-organization trust
- ✗ Slower ($O(n^2)$ messages)

PoA - Best for:

- ✓ High performance needed
- ✓ Trusted authority exists
- ✓ Private/consortium networks
- ✗ Single point of failure

Figure 33: Consensus - Recommandations d'utilisation

Analyse des cas d'usage optimaux pour chaque mécanisme :

- **PBFT recommandé pour** : Exigences de sécurité élevées, besoin de tolérance byzantine, environnements multi-organisationnels
- **PoA recommandé pour** : Besoins de performance élevée, autorité de confiance existante, réseaux privés/consortia
- **Vainqueur performance** : PoA (7 fois plus rapide)
- **Vainqueur sécurité** : PBFT (tolérant byzantin, décentralisé)

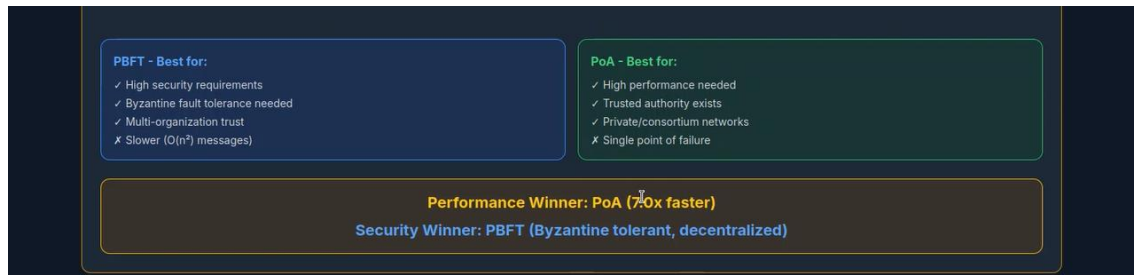


Figure 34: Consensus - Visualisation graphique des résultats

Représentation graphique des métriques de performance :

- Diagrammes comparatifs de débit (TPS) et latence (ms)
- Visualisation de l'évolutivité en fonction du nombre de nœuds
- Analyse du compromis performance/sécurité pour les applications Smart City*

7. Conclusion

Le projet **Traffic Core** représente une implémentation complète et fonctionnelle d'un **système de gestion de trafic urbain décentralisé** basé sur la technologie blockchain Hyperledger Fabric. À travers cette plateforme, nous avons démontré comment les **smart cities** peuvent bénéficier des avantages de la blockchain tout en intégrant des mécanismes de sécurité avancés et d'intelligence artificielle.

- **Réalisations Principales**

- **Architecture Robustes**

- Réseau Hyperledger Fabric opérationnel avec 3 organisations, 6 peers et 3 orderers
- Trois chaincodes CCaaS fonctionnels pour la gestion des routes, capteurs et registres
- Architecture microservices avec backend Express.js et frontend React

- **Sécurité Intelligente**

- SOC avec 4 agents IA spécialisés (Defender, Sensor, Analyzer, Controller)
- Intégration de Mistral 7B LLM pour l'analyse contextuelle des menaces
- Journalisation immuable des incidents sur la blockchain
- Simulation complète d'attaques cybernétiques (DDoS, Sybil, 51%, etc.)

- **Fonctionnalités Métier**

- Gestion dynamique de 10 routes et 8 intersections
- Détection automatique des violations avec calcul d'amendes selon la loi marocaine
- Tableau de bord temps réel avec rafraîchissement automatique
- Simulateur IoT complet avec scénarios réalistes (rush hour, conditions météo)

- **Résultats Techniques**

- **Performance** : Tests comparatifs PBFT vs PoA validant les compromis performance/sécurité
- **Scalabilité** : Architecture modulaire permettant l'ajout de capteurs et services
- **Interopérabilité** : API REST complète pour intégration avec d'autres systèmes

- **Expérience Utilisateur** : Interface intuitive avec navigation fluide entre 7 modules
- **Contribution Académique**
 - Ce projet démontre l'applicabilité concrète de la blockchain dans les infrastructures urbaines critiques, offrant :
 - Une alternative décentralisée aux systèmes centralisés traditionnels
 - Un modèle de gouvernance transparent entre multiples parties prenantes
 - Une approche innovante de sécurité proactive basée sur l'IA
- **Perspectives d'Évolution**
 1. **Intégration avec systèmes réels** : Connexion à des capteurs IoT physiques
 2. **Optimisation IA** : Amélioration des algorithmes de régulation du trafic
 3. **Élargissement du scope** : Ajout de modules énergie, stationnement, déchets
 4. **Tests à grande échelle** : Déploiement sur réseau de test étendu
- **Enseignements Clés**

Le développement de **Traffic Core** a validé plusieurs hypothèses importantes :

 - La blockchain permissionnée est adaptée aux environnements urbains sensibles
 - L'IA peut significativement renforcer la sécurité des systèmes critiques
 - La décentralisation améliore à la fois la résilience et la transparence
 - Une approche modulaire facilite l'évolution et la maintenance

Annexe A : Configuration et Déploiement du Réseau Blockchain

AXE 1 : Configuration du Réseau Hyperledger Fabric

1.1 Définition des Organisations

 Configuration OrdererOrg :

```
Organizations:
- &OrdererOrg

Name: OrdererMSP

ID: OrdererMSP

MSPDir: ../organizations/ordererOrganizations/traffic-core.example.com/msp

Policies:

  Readers: "OR('OrdererMSP.member')"

  Writers: "OR('OrdererMSP.member')"

  Admins: "OR('OrdererMSP.admin')"

  BlockValidation: "OR('OrdererMSP.member')"
```

 Configuration Org1 :

```
- &Org1

Name: Org1MSP

ID: Org1MSP

MSPDir: ../organizations/peerOrganizations/org1.traffic-core.example.com/msp

Policies:

  Readers: "OR('Org1MSP.admin', 'Org1MSP.peer', 'Org1MSP.client')"

  Writers: "OR('Org1MSP.admin', 'Org1MSP.client')"

  Admins: "OR('Org1MSP.admin')"

  Endorsement: "OR('Org1MSP.peer')"

AnchorPeers:

  - Host: peer0.org1.traffic-core.example.com

  Port: 7051
```

1.2. Configuration de l'Orderer (etcdraft)

Orderer: &OrdererDefaults

OrdererType: etcdraft

Addresses:

- orderer0.traffic-core.example.com:7050
- orderer1.traffic-core.example.com:8050
- orderer2.traffic-core.example.com:9050

EtcdRaft:

Consenters:

- Host: orderer0.traffic-core.example.com

Port: 7050

ClientTLSCert: [chemin vers server.crt]

ServerTLSCert: [chemin vers server.crt]

... (configurations similaires pour orderer1 et orderer2)

BatchTimeout: 2s

BatchSize:

MaxMessageCount: 500

AbsoluteMaxBytes: 10 MB

PreferredMaxBytes: 2 MB

1.3. Profils de Configuration des Canaux

Profiles:

TrafficCoreOrdererGenesis:

<<: *ChannelDefaults

Orderer:

<<: *OrdererDefaults

Organizations: [*OrdererOrg]

Consortiums:

TrafficCoreConsortium:

Organizations: [*Org1, *Org2, *Org3]

TrafficChannel:

```
<<: *ChannelDefaults  
  
Consortium: TrafficCoreConsortium  
  
Application:  
  
<<: *ApplicationDefaults  
  
Organizations: [*Org1, *Org2, *Org3]
```

AXE 2 : Génération des Artefacts du Réseau

2.1. Variables d'Environnement

```
export FABRIC_CFG_PATH=./config
```

2.2. Génération du Bloc de Genesis

```
configtxgen -profile TrafficCoreOrdererGenesis \  
-channelID system-channel \  
-outputBlock ./channel-artifacts/genesis.block
```

2.3. Création des Transactions de Canal

```
# Canal principal  
  
configtxgen -profile TrafficChannel \  
-outputCreateChannelTx ./channel-artifacts/traffic-channel.tx \  
-channelID traffic-channel
```

2.4. Configuration des Anchor Peers

```
# Org1  
  
configtxgen -profile TrafficChannel \  
-outputAnchorPeersUpdate ./channel-artifacts/Org1MSPanchors.tx \  
-channelID traffic-channel -asOrg Org1MSP  
  
# Org2  
  
configtxgen -profile TrafficChannel \  
-outputAnchorPeersUpdate ./channel-artifacts/Org2MSPanchors.tx \  
-channelID traffic-channel -asOrg Org2MSP  
  
# Org3  
  
configtxgen -profile TrafficChannel \  
-outputAnchorPeersUpdate ./channel-artifacts/Org3MSPanchors.tx \
```

```
-channelID traffic-channel -asOrg Org3MSP
```

AXE 3 : Déploiement et Gestion des Canaux

3.1. Variables d'Environnement pour l'Orderer

```
export  
ORDERER_CA=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/ordererOrganizations/traffic-core.example.com/orderers/orderer0.traffic-core.example.com/tls/ca.crt  
  
export ORDERER_URL=orderer0.traffic-core.example.com:7050
```

3.2. Création du Canal Principal

```
peer channel create -o $ORDERER_URL \  
-c traffic-channel \  
-f ./channel-artifacts/traffic-channel.tx \  
--tls --cafile $ORDERER_CA \  
--clientauth \  
--certfile $CORE_PEER_TLS_CLIENTCERT_FILE \  
--keyfile $CORE_PEER_TLS_CLIENTKEY_FILE \  
--outputBlock ./channel-artifacts/traffic-channel.block
```

3.3. Jointure des Peers au Canal

Commande de Jointure :

```
peer channel join -b ./channel-artifacts/traffic-channel.block
```

```
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_CLIENTAUTHREQUIRED=true  
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_CERT_FILE=$CORE_PEER_TLS_CLIENTCERT_FILE  
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_KEY_FILE=$CORE_PEER_TLS_CLIENTKEY_FILE  
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export FABRIC_CFG_PATH=/etc/hyperledger/fabric  
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer channel join -b ./channel-artifacts/traffic-channel.block  
[channelCmd] InitCmdFactory -> Endorser and orderer connections initialized  
[channelCmd] ExecuteJoin -> Successfully submitted proposal to join channel  
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer channel list  
Channels peers has joined:  
traffic-channel  
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer#
```

Vérification du Canal

```
# Lister les canaux  
  
peer channel list  
  
# Obtenir les informations du canal  
  
peer channel getinfo -c traffic-channel
```

```

root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer channel getinfo -c traffic-channel
[channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Blockchain info: {"height":1,"currentBlockHash":"rGa3aCA0VZbJK9xgtHEAXypPghgPlpI6U9BvEwQfQ2A="}
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer#

```

Chaque peer doit avoir ses variables d'environnement correctement configurées :

Exemple pour peer0.org2.traffic-core.example.com :

```

export CORE_PEER_ADDRESS=peer0.org2.traffic-core.example.com:9051
export CORE_PEER_LOCALMSPID=Org2MSP
export
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/p
eerOrganizations/org2.traffic-core.example.com/users/Admin@org2.traffic-core.example.com/msp
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_TLS_CLIENTAUTHREQUIRED=true
export
CORE_PEER_TLS_CLIENTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/organization
ns/peerOrganizations/org2.traffic-core.example.com/users/Admin@org2.traffic-
core.example.com/tls/client.crt
export
CORE_PEER_TLS_CLIENTKEY_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/organization
s/peerOrganizations/org2.traffic-core.example.com/users/Admin@org2.traffic-
core.example.com/tls/client.key
export
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/organization
s/peerOrganizations/org2.traffic-core.example.com/peers/peer0.org2.traffic-
core.example.com/tls/ca.crt

```

```

root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_ADDRESS=peer0.org2.traffic-c
ore.example.com:9051
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_LOCALMSPID=Org2MSP
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_MSPCONFIGPATH=/opt/gopath/sr
c/github.com/hyperledger/fabric/peer/organizations/peerOrganizations/org2.traffic-core.example.com/users/Admin@org2
.traffic-core.example.com/msp
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_ENABLED=true
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_CLIENTAUTHREQUIRED=true
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_CLIENTCERT_FILE=/opt/gop
ath/src/github.com/hyperledger/fabric/peer/organizations/peerOrganizations/org2.traffic-core.example.com/users/Admi
n@org2.traffic-core.example.com/tls/client.crt
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_CLIENTKEY_FILE=/opt/gopa
th/src/github.com/hyperledger/fabric/peer/organizations/peerOrganizations/org2.traffic-core.example.com/users/Admin
@org2.traffic-core.example.com/tls/client.key
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# export CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopat
h/src/github.com/hyperledger/fabric/peer/organizations/peerOrganizations/org2.traffic-core.example.com/peers/peer0.
org2.traffic-core.example.com/tls/ca.crt

```

Joindre le peer au channel

```

root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer channel join -b ./channel-artifacts/traf
fic-channel.block
[peerCmd] InitCmdFactory -> Endorser and orderer connections initialized
[peerCmd] InitCmdFactory -> Endorser and orderer connections initialized
[channelCmd] executeJoin -> Successfully submitted proposal to join channel
root@08c265e1447b:/opt/gopath/src/github.com/hyperledger/fabric/peer#

```

AXE 4 : Déploiement du Chaincode

4.1. Packaging du Chaincode

```
# Package avec label

peer lifecycle chaincode package traffic-core_v4.tar.gz \

--path /opt/gopath/src/github.com/chaincode/traffic-core \

--lang golang \

--label traffic-core_2.0

# Vérification

ls -l traffic-core_v2.tar.gz
```

4.2. Installation sur les Peers

Configuration requise avant chaque installation :

Exemple de Peer0 Org1 :

```
export CORE_PEER_ADDRESS=peer0.org1.traffic-core.example.com:7051
export CORE_PEER_LOCALMSPID=Org1MSP
export
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/
peerOrganizations/org1.traffic-core.example.com/users/Admin@org1.traffic-core.example.com/msp
export
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizatio
ns/peerOrganizations/org1.traffic-core.example.com/peers/peer0.org1.traffic-
core.example.com/tls/ca.crt
export CORE_PEER_TLS_ENABLED=true
```

```
peer lifecycle chaincode install traffic-core_v2.tar.gz
```

```
root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer lifecycle chaincode
install traffic-core_v2.tar.gz
[cli.lifecycle.chaincode] submitInstallProposal -> Instal
led remotely: response:<status:200 payload:"\nQtraffic-core_2.0:f16b68e2f7105c2cd113c9d90600089
650be8fb9a13372cea96a5649b0116e28\022\020traffic-core_2.0" >
[cli.lifecycle.chaincode] submitInstallProposal -> Chainc
ode code package identifier: traffic-core_2.0:f16b68e2f7105c2cd113c9d90600089650be8fb9a13372cea
96a5649b0116e28
```

4.3. Approbation des Organisations

Exemple de Org1 :

```
export
ORDERER_CA=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/ordererOrganizat
ions/traffic-core.example.com/orderers/orderer0.traffic-core.example.com/tls/ca.crt
```

```
peer lifecycle chaincode approveformyorg \
  -o orderer0.traffic-core.example.com:7050 \
  --ordererTLSHostnameOverride orderer0.traffic-core.example.com \
  --channelID traffic-channel \
  --name traffic-core \
  --version 1 \
  --package-id traffic-core_1:11d532ce7e9c2b8efd03bcab56f36a3eef359e82a1d98f077f9faaca \
  --sequence 1 \
  --tls \
  --cafile $ORDERER_CA
```

```
root@8b39b1e72e1:/opt/gopath/src/github.com/hyperledger/fabric/peer# export ORDERER_CA=/opt/gopath/src/github.com/hyperledger/fabric/peer/organizations/ordererOrganizations/traffic-core.example.com/orderers/orderer0.traffic-core.example.com/tls/ca.crt
root@8b39b1e72e1:/opt/gopath/src/github.com/hyperledger/fabric/peer# ls $ORDERER_CA
root@8b39b1e72e1:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer lifecycle chaincode approveformyorg \
> -o orderer0.traffic-core.example.com:7050 \
> --ordererTLSHostnameOverride orderer0.traffic-core.example.com \
> --channelID traffic-channel \
> --name traffic-core \
> --version 1 \
> --package-id traffic-core_1:11d532ce7e9c2b8efd03bcab56f36a3eef359e82a1d98f077f9faaca \
> --sequence 1 \
> --tls \
> --cafile $ORDERER_CA
[chaincodeCmd] ClientWait -> txid [315ea7f933633875d664f42e447a3ea2d3f1bce985cbf99315db939c4134f4b] committed with status (VALID) at peer0.org1.traffic-core.example.com:7051
```

4.4. Vérification du Readiness Check

```
peer lifecycle chaincode checkcommitreadiness \
  --channelID $CHANNEL_NAME \
  --name $CC_NAME \
  --version $CC_VERSION \
  --sequence $CC_SEQUENCE \
  --output json
```

```
root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer lifecycle chaincode
checkcommitreadiness \
> --channelID $CHANNEL_NAME \
> --name $CC_NAME \
> --version $CC_VERSION \
> --sequence $CC_SEQUENCE \
> --output json
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true,
    "Org3MSP": true
  }
}
root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer#
```

4.5. Commit du Chaincode

```
peer lifecycle chaincode commit \
  -o orderer0.traffic-core.example.com:7050 \
  --ordererTLSHostnameOverride orderer0.traffic-core.example.com \
  --channelID traffic-channel \
  --name traffic-core \
  --version 4.0 \
  --sequence 3 \
  --tls \
```

```
--cafile $ORDERER_CA \
--peerAddresses peer0.org1.traffic-core.example.com:7051 \
--peerAddresses peer0.org2.traffic-core.example.com:9051 \
--peerAddresses peer0.org3.traffic-core.example.com:11051 \
--tlsRootCertFiles organizations/peerOrganizations/org1.traffic-
core.example.com/peers/peer0.org1.traffic-core.example.com/tls/ca.crt \
--tlsRootCertFiles organizations/peerOrganizations/org2.traffic-
core.example.com/peers/peer0.org2.traffic-core.example.com/tls/ca.crt \
--tlsRootCertFiles organizations/peerOrganizations/org3.traffic-
core.example.com/peers/peer0.org3.traffic-core.example.com/tls/ca.crt
```

```
root@8b39b1e72e21:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer lifecycle chaincode commit \
> -o orderer0.traffic-core.example.com:7050 \
> --ordererTLSHostnameOverride orderer0.traffic-core.example.com \
> --channelID traffic-channel \
> --name traffic-core \
> --version 1 \
> --sequence 1 \
> --tls \
> --cafile $ORDERER_CA \
> --peerAddresses peer0.org1.traffic-core.example.com:7051 \
> --peerAddresses peer0.org2.traffic-core.example.com:9051 \
> --peerAddresses peer0.org3.traffic-core.example.com:11051 \
> --tlsRootCertFiles organizations/peerOrganizations/org1.traffic-core.example.com/peers/peer0.org1.traffic-core.example.com/tls/ca.crt \
> --tlsRootCertFiles organizations/peerOrganizations/org2.traffic-core.example.com/peers/peer0.org2.traffic-core.example.com/tls/ca.crt \
> --tlsRootCertFiles organizations/peerOrganizations/org3.traffic-core.example.com/peers/peer0.org3.traffic-core.example.com/tls/ca.crt \
> [chaincodeCmd] [initial] -> txid [8b30712b181db037df8b74add92e49692bedb73a6da2dbf82e2c4a82075a0718] committed with status (VALID) at peer0.org1.traffic-
core.example.com:9051
> [chaincodeCmd] [initial] -> txid [8b30712b181db037df8b74add92e49692bedb73a6da2dbf82e2c4a82075a0718] committed with status (VALID) at peer0.org1.traffic-
core.example.com:7051
> [chaincodeCmd] [initial] -> txid [8b30712b181db037df8b74add92e49692bedb73a6da2dbf82e2c4a82075a0718] committed with status (VALID) at peer0.org3.traffic-
core.example.com:11051
```

4.6. Vérification du Commit

```
peer lifecycle chaincode querycommitted --channelID traffic-channel --name
traffic-core
```

Exemple de sortie :

```
root@8b39b1e72e21:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer life
cycle chaincode querycommitted --channelID traffic-channel --name traffic-core
Committed chaincode definition for chaincode 'traffic-core' on channel 'traffic
-channel':
Version: 1, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, App
rovals: [Org1MSP: true, Org2MSP: true, Org3MSP: true]
root@8b39b1e72e21:/opt/gopath/src/github.com/hyperledger/fabric/peer#
```

AXE 5 : Tests Fonctionnels du Chaincode

5.1. Tests de Gestion des Routes

Route R001 créée : {"id":"R001","type":"highway","capacity":1200,"currentLoad":0,...}

```
peer chaincode invoke \
> -o orderer0.traffic-core.example.com:7050 \
> --tls \
> --cafile $ORDERER_CA \
> -C traffic-channel \
> -n traffic-core \
```

```

> -c '{"function": "CreateRoad", "Args": ["R001", "highway", "1200", "0", "0", "1000", "0"]}' \
> --peerAddresses peer0.org1.traffic-core.example.com:7051 \
es $PEER0_> --tlsRootCertFiles $PEER0_ORG1_CA \
> --peerAddresses peer0.org2.traffic-core.example.com:9051 \
> --tlsRootCertFiles $PEER0_ORG2_CA \
> --peerAddresses peer0.org3.traffic-core.example.com:11051 \
> --tlsRootCertFiles $PEER0_ORG3_CA

```

```

root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer chaincode invoke \
> -o orderer0.traffic-core.example.com:7050 \
> --tls \
> --cafile $ORDERER_CA \
> -C traffic-channel \
> -n traffic-core \
> -c '{"function": "CreateRoad", "Args": ["R001", "highway", "1200", "0", "0", "1000", "0"]}' \
> --peerAddresses peer0.org1.traffic-core.example.com:7051 \
es $PEER0_> --tlsRootCertFiles $PEER0_ORG1_CA \
> --peerAddresses peer0.org2.traffic-core.example.com:9051 \
> --tlsRootCertFiles $PEER0_ORG2_CA \
> --peerAddresses peer0.org3.traffic-core.example.com:11051 \
> --tlsRootCertFiles $PEER0_ORG3_CA
2026-01-02 14:25:43.113 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke
successful. result: status:200

```

5.2. Requêtes de Consultation

```

peer chaincode query \
-C traffic-channel \
-n traffic-core \
-c '{"function": "GetRoad", "Args": ["R001"]}'

```

```

root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer chaincode query \
fffic-cha> -C traffic-channel \
> -n traffic-core \
> -c '{"function": "GetRoad", "Args": ["R001"]}'
{"id": "R001", "type": "highway", "capacity": 1200, "currentLoad": 0, "geometry": {"start": {"x": 0, "y": 0},
"end": {"x": 1000, "y": 0}}, "createdAt": "2026-01-02T14:25:42Z"}
root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer chaincode query \
> -C traffic-channel \
> -n traffic-core \
> -c '{"function": "GetAllRoads", "Args": []}'
[{"id": "R001", "type": "highway", "capacity": 1200, "currentLoad": 0, "geometry": {"start": {"x": 0, "y": 0},
"end": {"x": 1000, "y": 0}}, "createdAt": "2026-01-02T14:25:42Z"}]

```

Obtenir toutes les routes

```

peer chaincode query \
-C traffic-channel \
-n traffic-core \
-c '{"function": "GetAllRoads", "Args": []}'

```

```

root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer chaincode query \
> -C traffic-channel \
> -n traffic-core \
> -c '{"function":"GetAllRoads","Args":[]}'
[{"id":"R001","type":"highway","capacity":1200,"currentLoad":0,"geometry":{"start":{"x":0,"y":0},
"end":{"x":1000,"y":0}},"createdAt":"2026-01-02T14:25:42Z"}]

```

5.3. Tests de Gestion des Véhicules

Enregistrer un véhicule

```

peer chaincode invoke \
-o orderer0.traffic-core.example.com:7050 \
--tls --cafile $ORDERER_CA \
-C traffic-channel \
-n traffic-core \
-c '{"function":"RegisterVehicle","Args":["V123","car","Alice","60","R001","10","20"]}' \
--peerAddresses peer0.org1.traffic-core.example.com:7051 \
--tlsRootCertFiles $PEER0_ORG1_CA \
--peerAddresses peer0.org2.traffic-core.example.com:9051 \
--tlsRootCertFiles $PEER0_ORG2_CA \
--peerAddresses peer0.org3.traffic-core.example.com:11051 \
--tlsRootCertFiles $PEER0_ORG3_CA

```

```

root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer chaincode invoke \
> -o orderer0.traffic-core.example.com:7050 \
--tls \> --tls \
> --cafile $ORDERER_CA \
> -C traffic-channel \
> -n traffic-core \
> -c '{"function":"RegisterVehicle","Args":["V123","car","Alice","60","R001","10","20"]}' \
> --peerAddresses peer0.org1.traffic-core.example.com:7051 \
> --tlsRootCertFiles $PEER0_ORG1_CA \
> --peerAddresses peer0.org2.traffic-core.example.com:9051 \
> --tlsRootCertFiles $PEER0_ORG2_CA \
> --peerAddresses peer0.org3.traffic-core.example.com:11051 \
> --tlsRootCertFiles $PEER0_ORG3_CA
2026-01-02 14:26:15.035 UTC [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke
successful. result: status:200

```

Consulter un véhicule

```

peer chaincode query \
-C traffic-channel \
-n traffic-core \
-c '{"function":"GetVehicle","Args":["V123"]}'

```

```

root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# peer chaincode query \
> -C traffic-channel \
> -n traffic-core \
> -c '{"function":"GetVehicle","Args":["V123"]}'
{"id":"V123","type":"car","owner":"Alice","speed":60,"timestamp":"2026-01-02T14:28:15Z","position":{"roadID":"R001","x":10,"y":20}}

```

5.4. Vérification de la Base de Données

```
curl -u admin:adminpw "http://couchdb0:5984/traffic-channel_traffic-core/_all_docs?include_docs=true"
```

```

root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer# curl -u admin:adminpw "http://couchdb0:5984/traffic-channel_traffic-core/_all_docs?include_docs=true"
{"total_rows":5,"offset":0,"rows":[{"_id":"ROAD:R001","key":"ROAD:R001","value":{"rev":"2-b3841304c378122ef51b5ab5144cb029","doc":{"_id":"ROAD:R001","_rev":"2-b3841304c378122ef51b5ab5144cb029","capacity":1200,"createdat":"2026-01-02T14:25:42Z","currentLoad":1,"geometry":{"end":{"x":1000,"y":0},"start":{"x":0,"y":0},"id":"R001","type":"highway","version":"CgMBHQA="},"_id":"VEHICLE:V123","key":"VEHICLE:V123","value":{"rev":"1-308920dfdc4791817fde26cd9c9db964b","doc":{"_id":"VEHICLE:V123","_rev":"1-308920dfdc4791817fde26cd9c9db964b","id":"V123","owner":"Alice","position":{"roadID":"R001","x":10,"y":20},"speed":60,"timestamp":"2026-01-02T14:28:15Z","type":"car","version":"CgMBHQA="},"_id":"_design/indexMovementDoc","key":"_design/indexMovementDoc","value":{"rev":"1-c015b00488a48f0898cc70f12a65dee","doc":{"_id":"_design/indexMovementDoc","_rev":"1-c015b00488a48f0898cc70f12a65dee","language":"query","views":{"indexMovement":{"map":{"fields":{"vehicleID":"asc","timestamp":"asc","oldPos.roadID":"asc","newPos.roadID":"asc"},"partial_filter_selector":{},"reduce":{"_count"},"options":{"fields":{"vehicleID","timestamp","oldPos.roadID","newPos.roadID"}}}}}}}},"_id":"_design/indexRoadDoc","key":"_design/indexRoadDoc","value":{"rev":"1-ab3c5b1b958cbf0279d659442280dc8be","doc":{"_id":"_design/indexRoadDoc","_rev":"1-ab3c5b1b958cbf0279d659442280dc8be","language":"query","views":{"indexRoad":{"map":{"fields":{"id":"asc","type":"asc","currentLoad":"asc","capacity":"asc"},"partial_filter_selector":{},"reduce":{"_count"},"options":{"def":{"fields":{"id","type","currentLoad","capacity"}}}}}}}},"_id":"_design/indexVehicleDoc","key":"_design/indexVehicleDoc","value":{"rev":"1-dd57d79811c8c14cbf-f1179ba138b37","doc":{"_id":"_design/indexVehicleDoc","_rev":"1-dd57d79811c8c14cbf-f1179ba138b37","language":"query","views":{"indexVehicle":{"map":{"fields":{"id":"asc","type":"asc","timestamp":"asc","position.roadID":"asc"},"partial_filter_selector":{},"reduce":{"_count"},"options":{"def":{"fields":{"id","type","timestamp","position.roadID"}}}}}}}}}}]}]}
root@2be45f4db3e8:/opt/gopath/src/github.com/hyperledger/fabric/peer#

```